



Università della Calabria

**Dipartimento di Ingegneria Informatica, Modellistica,
Elettronica e Sistemistica**

Master Degree in Robotics And Automation Engineering

Course: Industrial Automation

Control Of A CNC-Machine

Student

Alessandro Furina
252328

Professor

Pietro Maria Muraca

Academic Year 2023-2024

TABLE OF CONTENTS

Chapter 1: [Introduction](#)

Chapter 2: [Reference Signal](#)

Chapter 3: [Control Unit](#)

Chapter 4: [Finite State Machine](#)

[Appendix](#)

INTRODUCTION

The aim of the project consists of performing a cutting task of specific figures by using a CNC Machine (Computer Numerical Control). A CNC machine is an operating machine able to complete different typical industrial operations such as: cutting, assembling and making tasks which require a high precision level. The structure typically is composed of the mechanical part that represents the geometry of the machine in terms of shape and dimension; this component allow the movement of the body by translation or rotational joints (each joint represents one degree of freedom). Moreover, there is the terminal part of the machine, commonly said End Effector, projected to perform the specific task required; thus, this mechanical component can be modified in order to adapt the machine to the operating application. Usually, it adds further degree of freedom to the structure. The following figure shows an example of a modern CNC machine:



Fig. 1.1: Example of CNC Machine

We can appreciate the two main units of the machine:

1. The working station: dedicated to perform the operation task and described so far.
2. The control unit: responsible for working station's management generating the control signals for the actuators.

Under an abstract point of view the control unit could be identified as the 'brain' of the machine that acts by the 'arm' (the working station).

This work is oriented to project the main jobs of the control unit which can be summed up as follows:

1. Computation of the reference signals to design the paths.
2. Computation of the actuator signals to perform the task.

In order to allow the movement of the joints we have to introduce the last main component of a typical CNC machine that is the Motor. It is an electro-mechanical part able to convert electrical energy into mechanical energy and vice versa. In commerce there are a lot of Motor developed for

different jobs, however our attention is focused on a particular class of them that are the DC Motor (Direct Current Motor). In a schematic way we can illustrate them as follows:

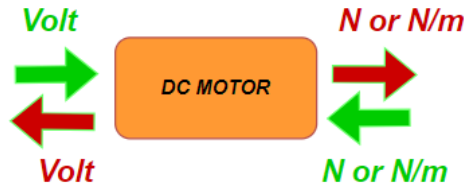


Fig. 1.2: Diagram of DC Motor

In the context of interest, we want to put our attention on the first conversion path from electrical to mechanical energy. In fact, from the control unit the DC motor receives the signal in the form of digital input, so electrical input, that it must convert in movements of the joints, so mechanical output. The general illustration is the following:

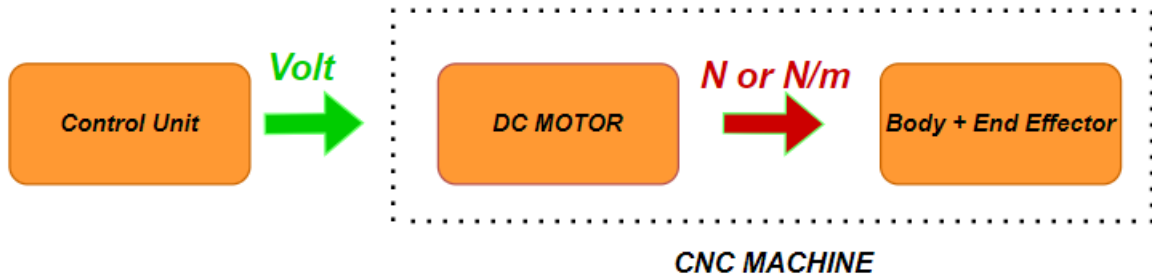


Fig. 1.3: Diagram of CNC Machine

As we asserted before, the core of the project consists in designing appropriately the reference signals for the control unit and, consequently, the actuator signals for the DC Motor to obtain the results. So, it's important to understand physically how DC Motor are typically maken and their dynamic. Usually, they are composed of an electrical circuit for the electrical part and of a rotational beam for the mechanical part:

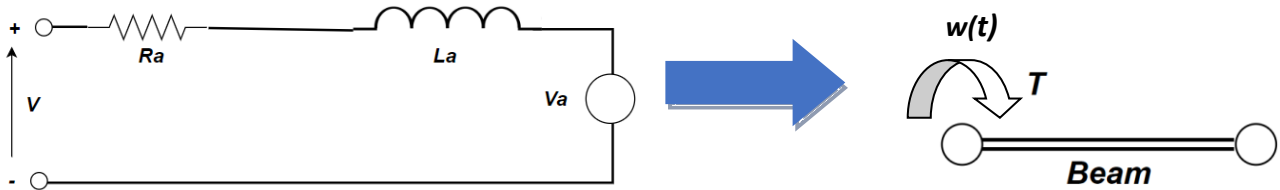


Fig. 1.4: Physical diagram of DC Motor

The electrical energy is converted into heat and mechanical rotational energy of the beam. The evolution of the motor can be summed up with the following laws:

$$\begin{cases} V(t) = R_a * i(t) + L_a * \frac{di(t)}{dt} + V_a(t) \\ J * \frac{dw(t)}{dt} + B * w(t) = T(t) \\ T(t) = K_t * i(t) \\ V_a(t) = K_e * w(t) \end{cases}$$

With:

- R_a, L_a, i, V_a : respectively values of resistor, inductor, current and voltage generator.
- J : inertia of the motor
- B : friction coefficient of the motor
- W : angular velocity of the beam due to the torque T generated by the electrical part
- K_t : conversion coefficient from electrical energy into mechanical energy
- K_e : Lenz law coefficient

Applying the Laplace Transform to the system we obtain the block representation of the motor in the s domain:

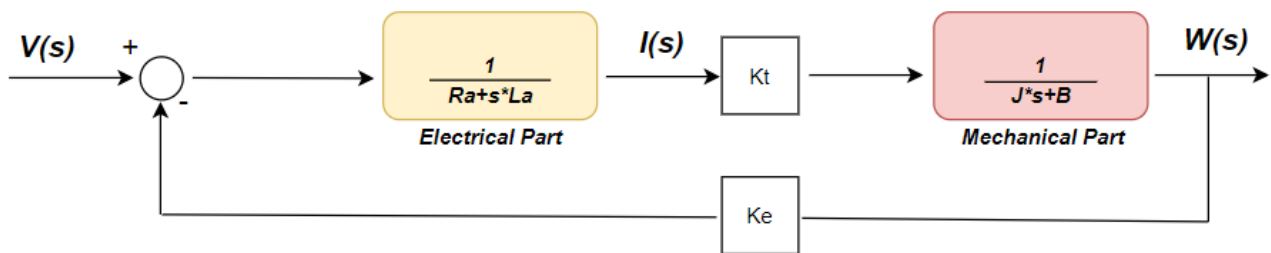


Fig. 1.5: Block Representation of DC Motor

With the values $\frac{-R_a}{L_a}$ and $\frac{-B}{J}$ representing respectively the poles of the electrical and mechanical part.

Once we described the dynamic of the DC Motor, we can go on explaining the main topics of the work.

As introduced above, we are interested in a cut performing of specific figures. In particular, these are: a triangle, a circle and a half-circle. The operating area is a 3 by 2 meters area with the figures located in this way:

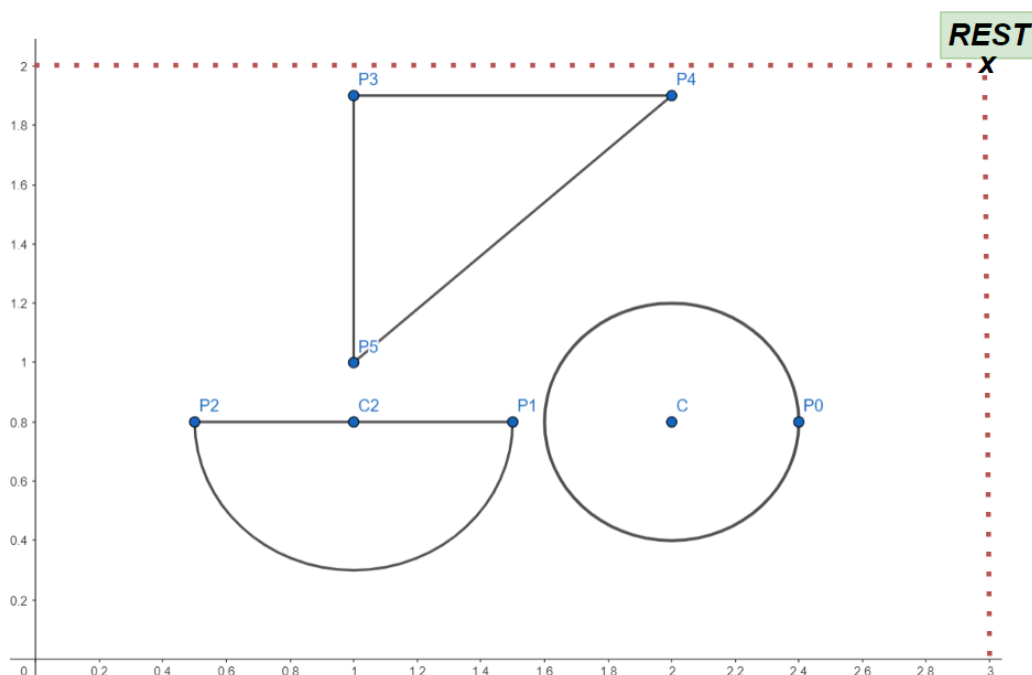


Fig. 1.6: Operative Area

The initial and final position of the machine is set in the Rest point (3;2), thus at the beginning and at the end of the task the machine is located there.

Because of a single motor can guarantee a movement just in one direction, to define movements in 2D, as the task requires, we will use two DC Motor: one for the displacement along x direction and the other for the displacement along y direction. However, we need a mechanism to convert rotational movement of the Motor into translational movement of the machine; in general, this aim is reached using a wheel-rack mechanism in which a teeth wheel rotates with a certain angular velocity on a rack which can operate translational movements. The rotation of the wheel by the contact with the rack is transformed into translational velocity of the rack, so into linear velocity.

Now we can introduce the main characteristics of the two identical DC motor we will use:

- $R_a = 1 \text{ } [\Omega]$
- $L_a = 1.0 \text{ [mH]}$
- $J = 0.12$
- $B = 0.03 \text{ (motor + load)}$
- $K_e = 0.6 \text{ [Volts*sec/rad]}$
- $K_t = 0.6 \text{ [N*m/A]}$
- Max power = 4 [KW]
- Max Armature current = 30 [A]
- Max Voltage = 200 [V]
- The mechanism that transforms the rotation into translation has a transformation coefficient equal to $1 \text{ [m]} = 800 \text{ [rev]}$

For each motor it is possible to measure the armature current, the rotational speed [rad/sec] and the angle [rad].

The requirements of the task are the following:

- maximum position error of less than 2 [mm] even when there is an uncertainty of parameters J and B of 10% with respect to the nominal ones.
- current, voltage and power within the range described above.
- development of a finite state machine and its possible implementation (Ladder and or SFC) for machine management.

The project consists of two phases:

1. Creation of the reference signals for x and y directions
2. Design of the control unit to follow the path from the reference satisfying the characteristics described.

First part is strictly related to the approach chosen for the second one. In this work we use a control design method based on the realization of three controllers: one for the position, one for the angular speed and the last one for the current. The design of each one can be realized independently based on the different dynamics of the controllers. To have an idea of this, it is sufficient to remember that the settling time T_s is:

$$T_s \cong 4\tau$$

With τ time constant of the system and equal to:

$$\tau = \frac{1}{|dominantPole|}$$

So, in the scenario of interest we have:

$$\tau_e = \frac{La}{Ra} = 10^{-3} [sec] \quad \tau_m = \frac{J}{B} = 4 [sec]$$

Because of there is an important difference between electrical and mechanical time constant we can design the controllers separately.

REFERENCE SIGNAL

The design of the reference signals takes into account the scenario illustrated in Fig. 1.6. The sequence chosen for the operations is the following:

1. Move from Rest position to P0.
2. Cut the circular shape.
3. Move from P0 to P1.
4. Cut the half circular shape.
5. Cut from P2 to P1.
6. Move from P1 to P5.
7. Cut the triangular shape (P5→P3→P4→P5)
8. Move from P5 to the Rest position.

For the design of the reference a polynomial approach was used. In fact, during a path between two points we have to guarantee that the initial and final position coincide with the points considered, but also we need the conditions of velocity and acceleration equal to zero when the machine reaches these points. This is important to avoid sudden variations of the velocity and acceleration profile. Thus, we end up with:

$$\begin{cases} x(t_0) = x_0 \\ x(t_f) = x_f \\ v(t_0) = 0 \\ v(t_f) = 0 \\ a(t_0) = 0 \\ a(t_f) = 0 \end{cases}$$

Because of there are six conditions, we have to use a 5-order polynomial function to satisfy them.

To transform the problem of path planning in a parametric way we convert the time interval $[t_0, t_f]$ into the normalized interval $[0, 1]$ and we do the same for the x interval $[x_0, x_f]$. This trick is useful to solve the problem in the square domain, then we will map the result in the original domains. So, now let's consider the polynomial function $\lambda(\sigma)$:

$$\lambda: [0,1] \rightarrow [0,1]$$

Such that:

$$\left\{ \begin{array}{l} \lambda(0) = 0 \\ \lambda(1) = 1 \\ \frac{d\lambda(\sigma)}{d\sigma}(0) = 0 \\ \frac{d\lambda(\sigma)}{d\sigma}(1) = 0 \\ \frac{d^2\lambda(\sigma)}{d^2\sigma}(0) = 0 \\ \frac{d^2\lambda(\sigma)}{d^2\sigma}(1) = 0 \end{array} \right.$$

Considering the 5-order polynomial function:

$$\lambda(\sigma) = \alpha_0 + \alpha_1 * \sigma + \alpha_2 * \sigma^2 + \alpha_3 * \sigma^3 + \alpha_4 * \sigma^4 + \alpha_5 * \sigma^5$$

The solution of the above system is:

$$\lambda(\sigma) = 10 * \sigma^3 - 15 * \sigma^4 + 6 * \sigma^5$$

The following Matlab code is responsible for the x path reference.

First of all, let's store the data of interest:

```
% X Reference Generator:
Cx=2;%center of the circle [x coordinate]
P0=2.4;%point of the circle [x coordinate]
r=0.4;%radius of the circle [m]
P1=1.5;%extreme point of the half circle [x coordinate]
P2=0.5;%extreme point of the half circle [x coordinate]
C2x=1;%center of the half circle [x coordinate]
r2=0.5;%radius of the half circle [m]
P5=1;%vertex of the triangle [x coordinate]
P3=1;%vertex of the triangle [x coordinate]
P4=2;%vertex of the triangle [x coordinate]
xrest=3;%rest position [x coordinate]
deltat=80;%time interval for each path
```

Secondary, we can parameterize the first path:

```

%Segment from rest to P0: Laser Off
t0=0;t1=deltat;
sigma=[0:0.1/80:1];
t_1=t0+sigma*(t1-t0);
x0=xrest;x1=P0;
sigma_star=(t_1-t0)/(t1-t0);
lambda_star=poly5(sigma_star);
x_1=x0+lambda_star*(x1-x0);
t=t_1;

```

The time interval was divided into small interval of 0.1 [sec]. Then, to parameterize the path, the variable 'lambda_star' that associates each sigma value with the corresponding value of the 5-order function 'poly5' was used, this one implements the polynomial described before. At the end, the parameterization was reconverted into the space domain.

The rest of the code uses the same approach to complete the path parameterization:

```

%Circular shape: Laser On
sigma=[0.1/80:0.1/80:1];
t2=0;t3=deltat;
t_2=t2+sigma*(t3-t2);
sigma_star=(t_2-t2)/(t3-t2);
lambda_star=poly5(sigma_star);
x_2=Cx+r*cos(lambda_star*2*pi);
t=[t,t_2+t(end)];
%Segment from P0 to P1: Laser Off
t4=0;t5=deltat;
t_3=t4+sigma*(t5-t4);
sigma_star=(t_3-t4)/(t5-t4);
x2=P0;x3=P1;
lambda_star=poly5(sigma_star);
x_3=x2+lambda_star*(x3-x2);
t=[t,t_3+t(end)];

```

```

%Half circular shape:Laser On
t6=0;t7=deltat;
t_4=t6+sigma*(t7-t6);
sigma_star=(t_4-t6)/(t7-t6);
lambda_star=poly5(sigma_star);
x_4=C2x+r2*cos(lambda_star*pi);
t=[t,t_4+t(end)];
%Segment from P2 to P1:Laser On
t8=0;t9=deltat;
t_5=t8+sigma*(t9-t8);
sigma_star=(t_5-t8)/(t9-t8);
x4=P2;x5=P1;
lambda_star=poly5(sigma_star);
x_5=x4+lambda_star*(x5-x4);
t=[t,t_5+t(end)];
%Segment from P1 to P5: Laser OFF
t10=0;t11=deltat;
t_6=t10+sigma*(t11-t10);
sigma_star=(t_6-t10)/(t11-t10);
x6=P1;x7=P5;
lambda_star=poly5(sigma_star);
x_6=x6+lambda_star*(x7-x6);
t=[t,t_6+t(end)];
%Segment from P5 to P3: Laser ON
t12=0;t13=deltat;
t_7=t12+sigma*(t13-t12);
sigma_star=(t_7-t12)/(t13-t12);
x8=P5;x9=P3;
lambda_star=poly5(sigma_star);
x_7=x8+lambda_star*(x9-x8);
t=[t,t_7+t(end)];

```

```

%Segment from P3 to P4: Laser ON
t14=0;t15=deltat;
t_8=t14+sigma*(t15-t14);
sigma_star=(t_8-t14)/(t15-t14);
x10=P3;x11=P4;
lambda_star=poly5(sigma_star);
x_8=x10+lambda_star*(x11-x10);
t=[t,t_8+t(end)];
%Segment from P4 to P5: Laser ON
t16=0;t17=deltat;
t_9=t16+sigma*(t17-t16);
sigma_star=(t_9-t16)/(t17-t16);
x12=P4;x13=P5;
lambda_star=poly5(sigma_star);
x_9=x12+lambda_star*(x13-x12);
t=[t,t_9+t(end)];

%Segment from P5 to rest: Laser OFF
t18=0;t19=deltat;
t_10=t18+sigma*(t19-t18);
sigma_star=(t_10-t18)/(t19-t18);
x14=P5;x15=xrest;
lambda_star=poly5(sigma_star);
x_10=x14+lambda_star*(x15-x14);
t=[t,t_10+t(end)];
x=[x_1,x_2,x_3,x_4,x_5,x_6,x_7,x_8,x_9,x_10];

```

After the operations, vector x contains the values of the path corresponding to each time instants stored in the vector t. The code for the y reference is almost equal to the one above, except for the data associated to the points which consider the y position in lieu of x position. The code is attached in the appendix.

Now what we have to do is the generation of the signals in Simulink in order to use them as sources for the motors. The following figure illustrates the Simulink blocks responsible for the creation of the signal x starting from vectors x and t:

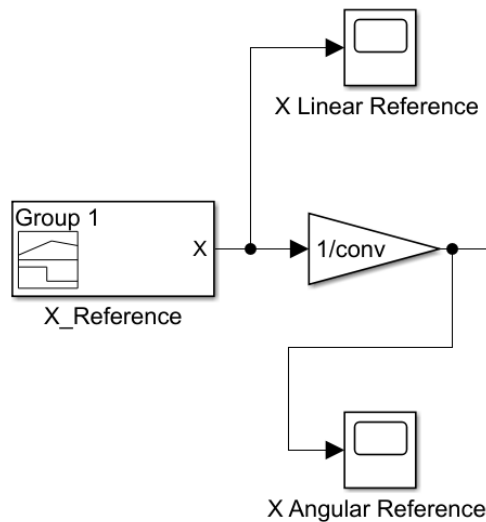


Fig. 2.1: X Signal Generator (Simulink)

The Signal Builder block generates the linear reference and, because of the motor works with angular position, it is converted into angular reference by the transformation coefficient ' $conv' = 1/(2 \cdot \pi \cdot 800) \text{ [rev]}^{-1}$ '. The same approach is used to create the y reference taking into account vector y instead of vector x. The evolutions of the signals are the following:

X LINEAR:

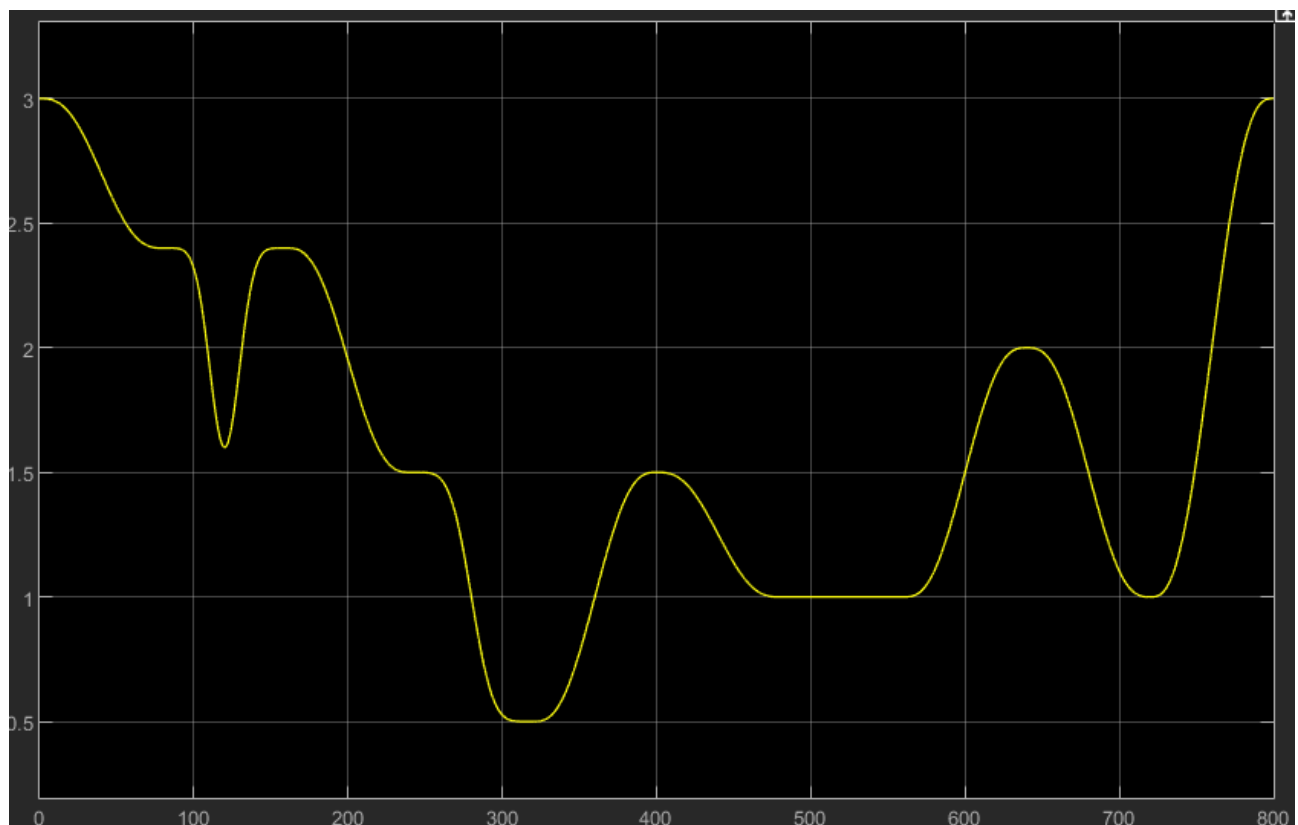


Fig. 2.2: X Linear Reference (Simulink)

X ANGULAR:

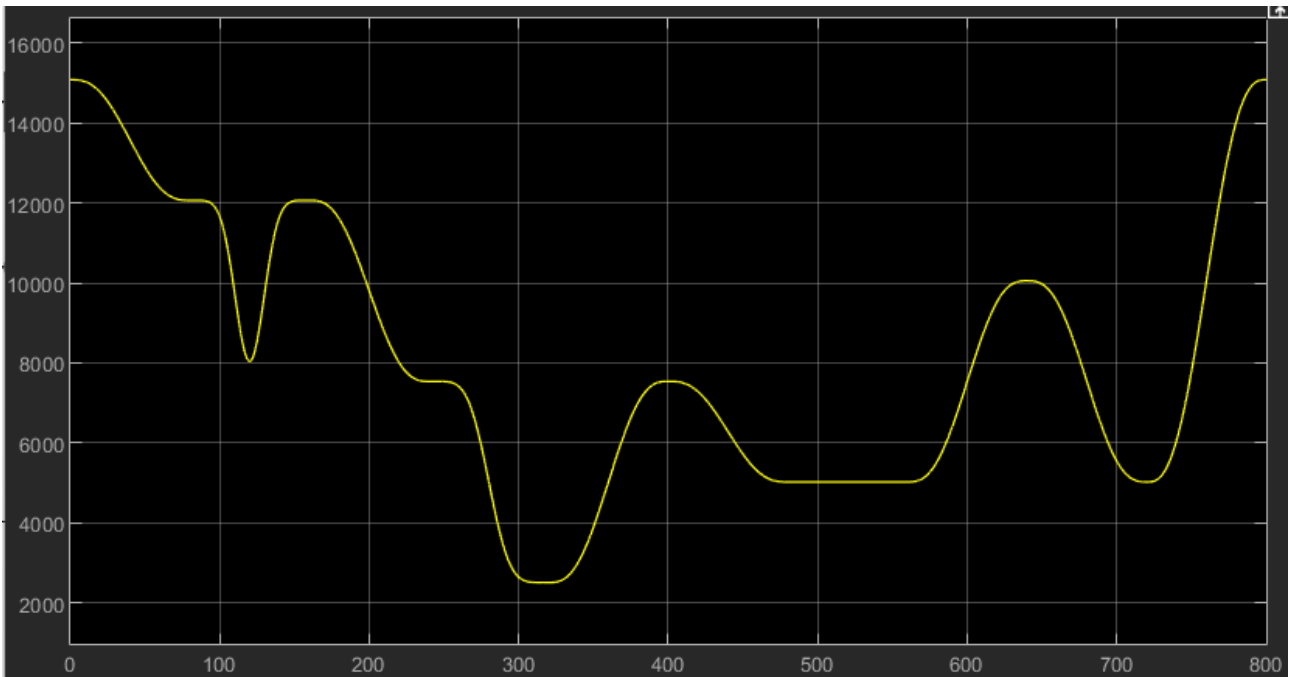


Fig. 2.3: X Angular Reference (Simulink)

Y LINEAR:

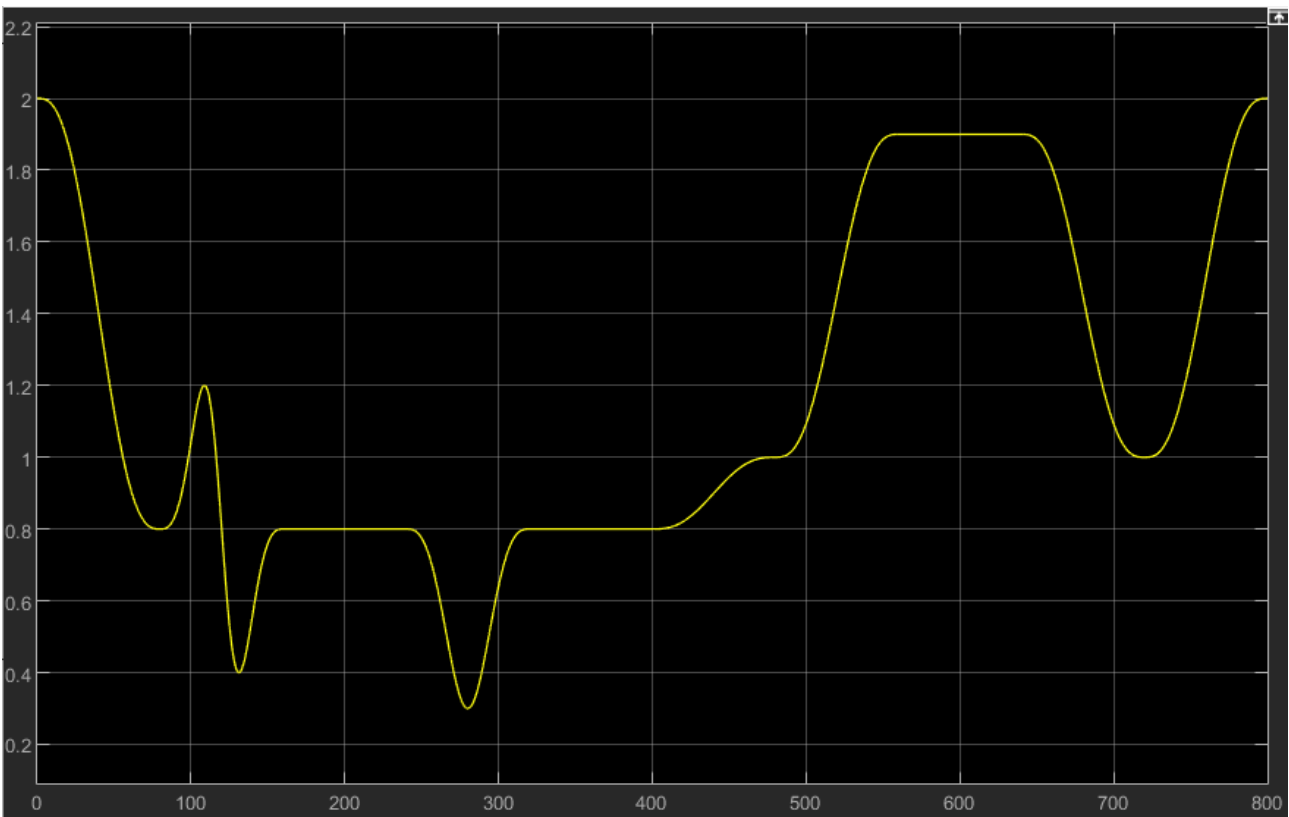


Fig. 2.4: Y Linear Reference (Simulink)

Y ANGULAR:

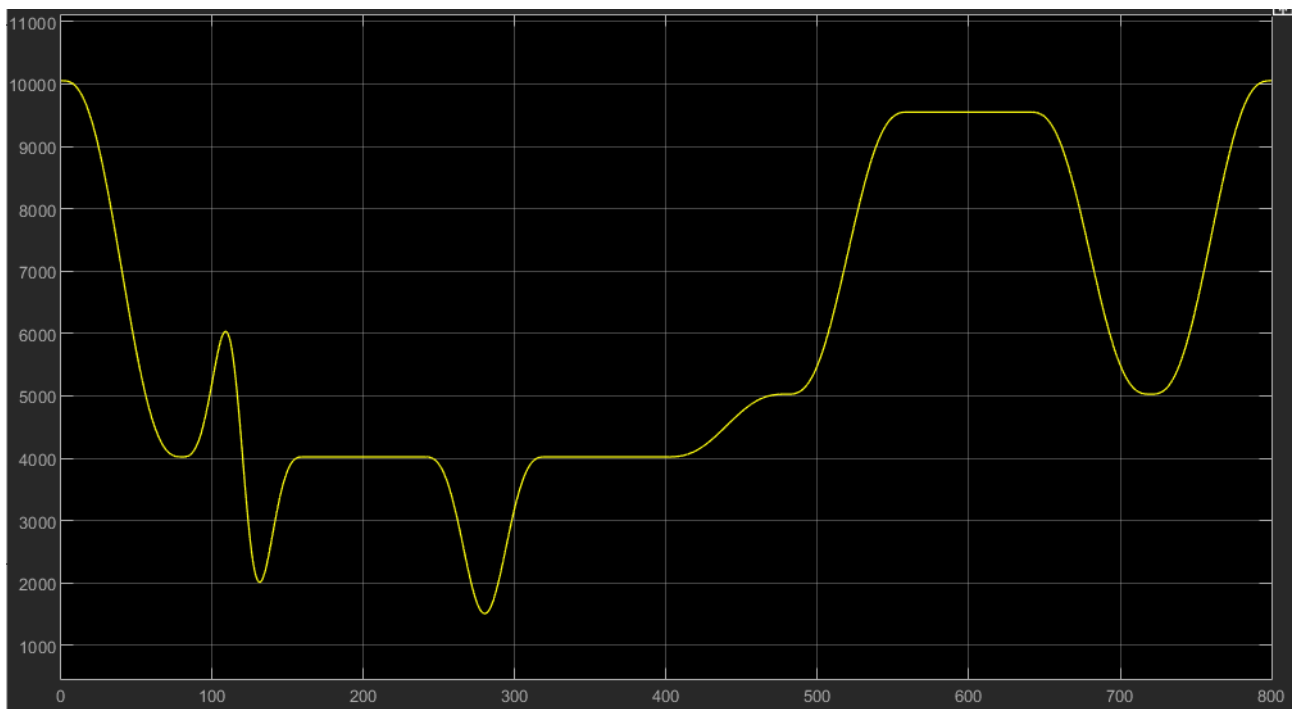


Fig. 2.5: Y Angular Reference (Simulink)

We can appreciate the smoothness of the signals thanks to the use of a 5-order polynomial function.

CONTROL UNIT

The design of the control unit is based on the idea of the three controllers (position, angular speed and current) discussed previously. It's possible to project each controller without considering the presence of the others because the dynamic of each one is quite different in terms of speed. The controllers will complete the control task by using feedback between the reference and the current value of the quantity they are responsible for.

First, let's implement the DC motor scheme in Simulink:

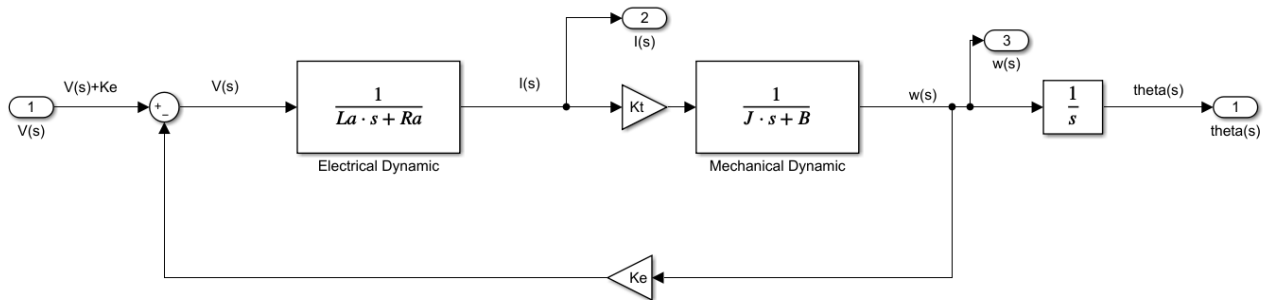


Fig. 3.1: DC Motor (Simulink)

The parameters of the motor are stored in a Matlab file called 'Parameters'. The input of the Motor is the voltage from the current controller while the outputs are the current, the angular speed and the angle generated by the rotational beam.

The entire system composed by Signal Generators, Control Units and DC motors is illustrated below:

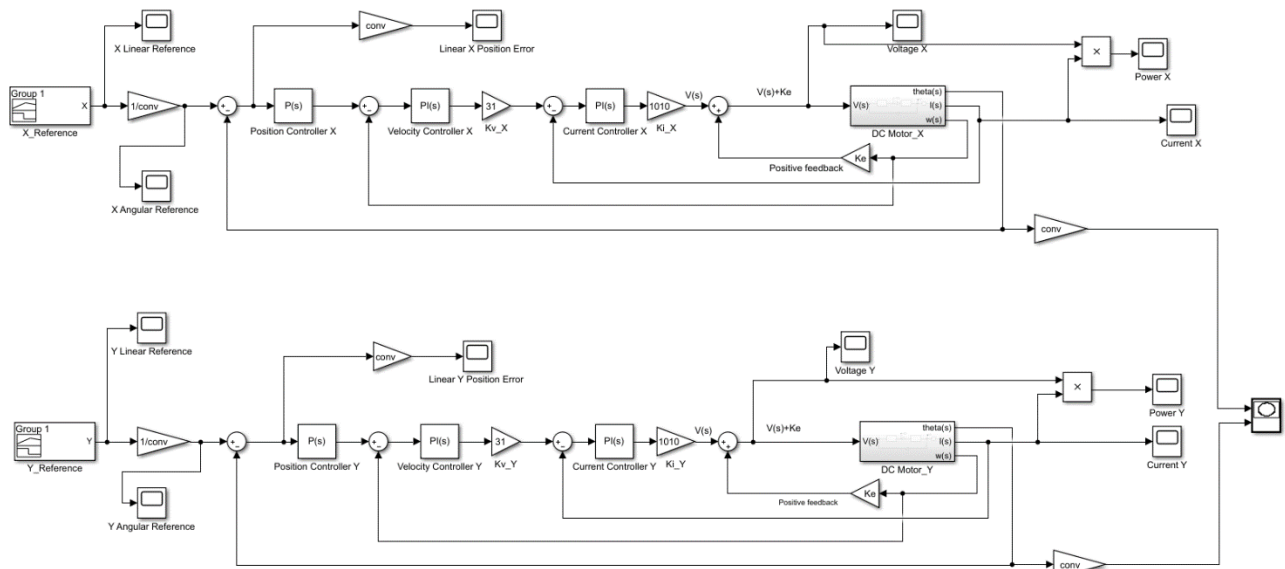


Fig. 3.2: CNC Machine System (Simulink)

To be clearer in the discussion we are referring to just one control system, for example x position. The topics will be explained are the same in the case of y position.

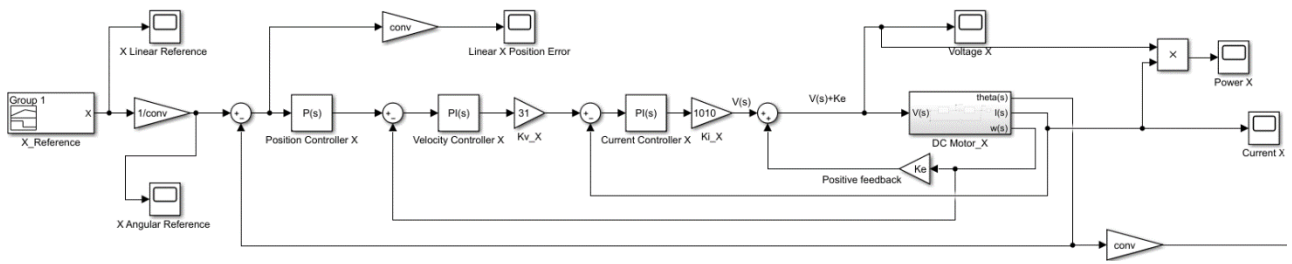


Fig. 3.3: X DC Motor System (Simulink)

Positive feedback with the value K_e is applied to the angular speed of the motor. This is realized to eliminate the internal feedback of the motor, so the corresponding electromotive force $k_e \cdot w(t)$. In this case, because of the value k_e is known, positive feedback is accurate and, thus, the system doesn't get instability. Otherwise, if k_e was unknown we should estimate its range of operation and take the lower bound in order to stay safe. The control unit is represented by the sequence of the three controllers:

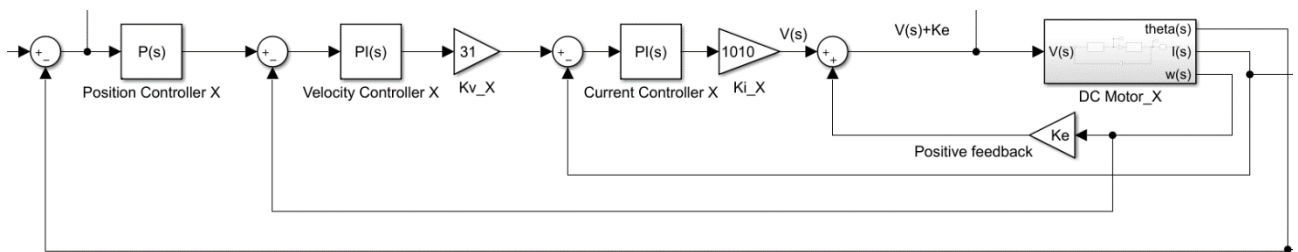


Fig. 3.4: X DC Motor System focused (Simulink)

They were chosen as it follows:

- Proportional controller for the position with gain $K_p=30$
- PI (Proportional-Integral) controller for the speed with parameters:
 1. $K_p=J$
 2. $K_i=B$
 3. Gain $K_{v_x}=31$

The choice of K_p and K_i allows to eliminate the dynamic of the motor's mechanical part.

- PI (Proportional-Integral) controller for the current with parameters:
 1. $K_p=La$
 2. $K_i=Ra$
 3. Gain $K_{i_x}=1010$

The choice of K_p and K_i allows to eliminate the dynamic of the motor's electrical part.

The gain K_p, K_{v_x}, K_{i_x} are chosen to guarantee the dynamic of the current to be faster than the dynamic of the speed that is faster than the dynamic of the position.

Now we can analyse the results in terms of linear position error, voltage, current and power:

X LINEAR POSITION ERROR:

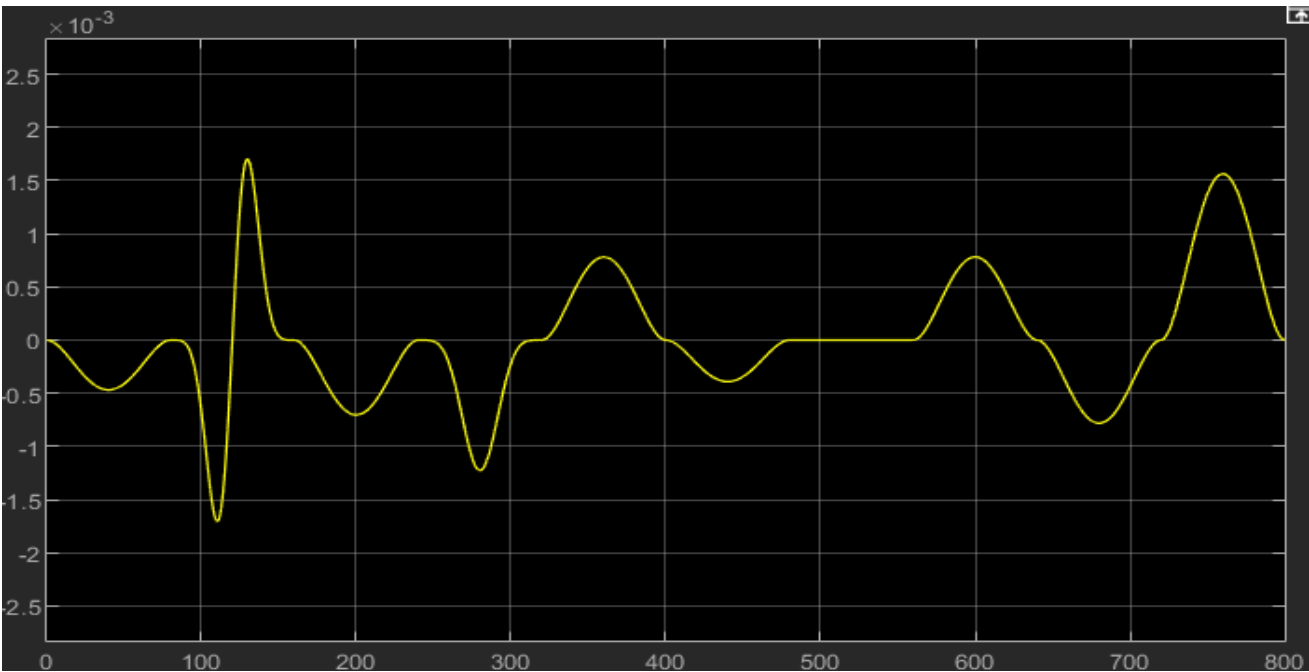


Fig. 3.5: X Linear Position Error (Simulink)

X VOLTAGE:

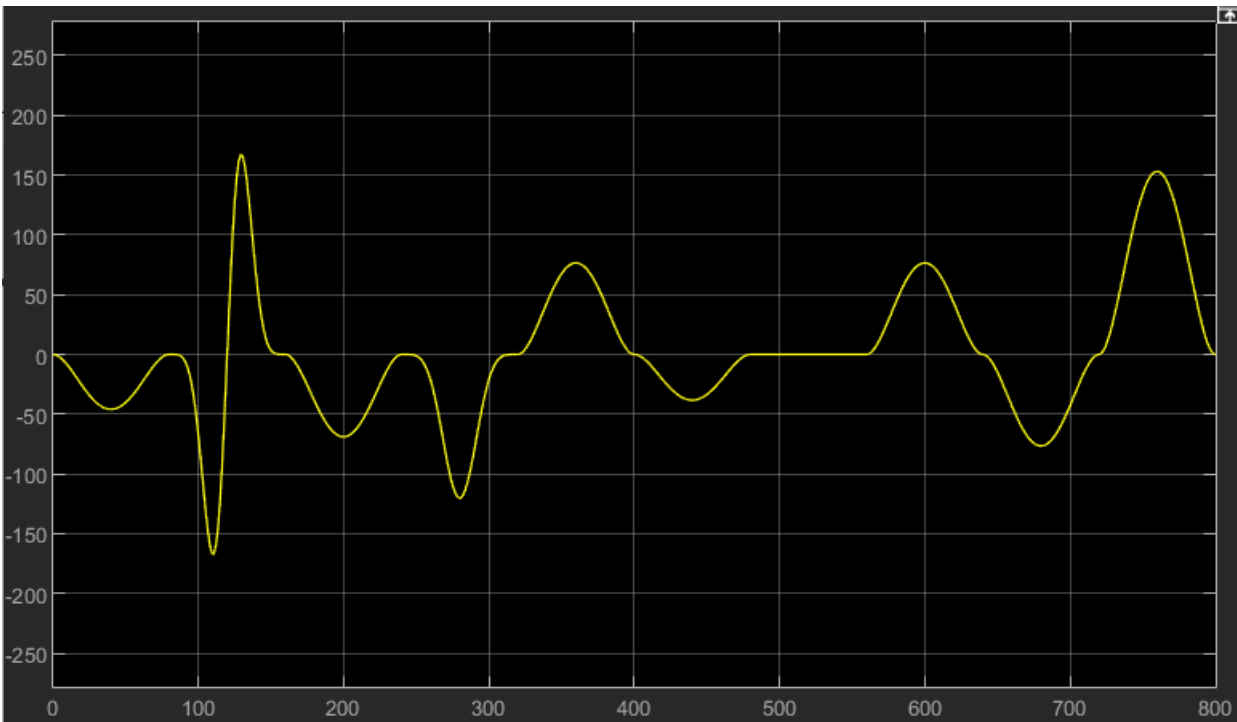


Fig. 3.6: X DC Motor Voltage (Simulink)

X CURRENT:

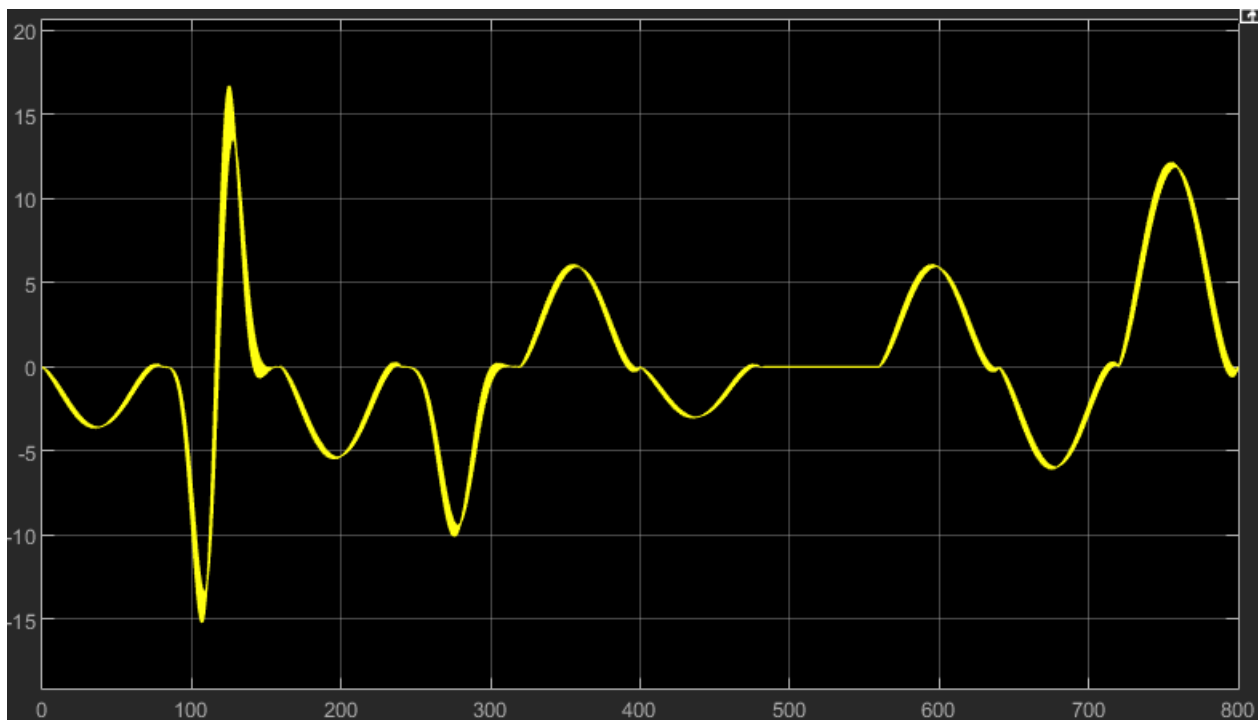


Fig. 3.7: X DC Motor Current (Simulink)

X POWER:

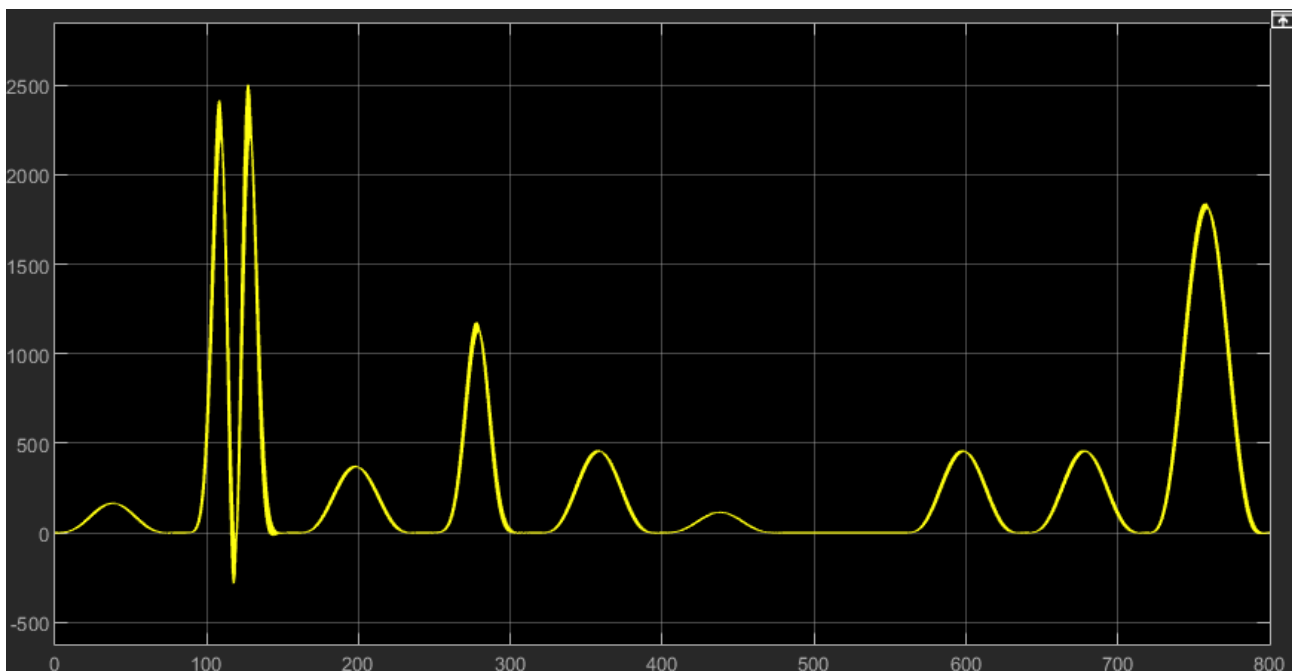


Fig. 3.8: X DC Motor Power (Simulink)

In nominal condition the requirements about linear error position under 2 [mm], max Voltage under 200 [Volt], max Current under 30 [A] and max Power under 4 [kW] are satisfied. The same happens for the motor responsible for y position. The results are attached in the appendix.

The XY Graph receives as inputs the x and y positions of the machine. The output is the following:

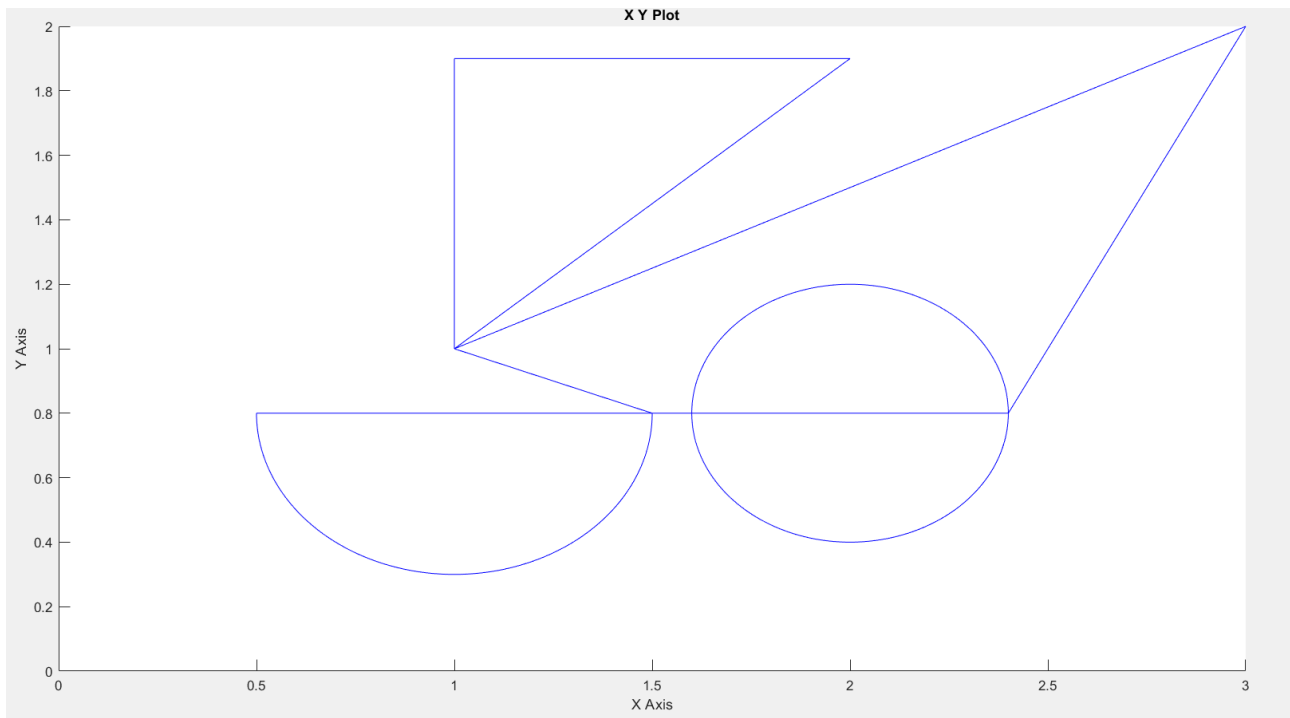


Fig. 3.9: XY Graph (Simulink)

The machine completed the task following the sequence reported below:

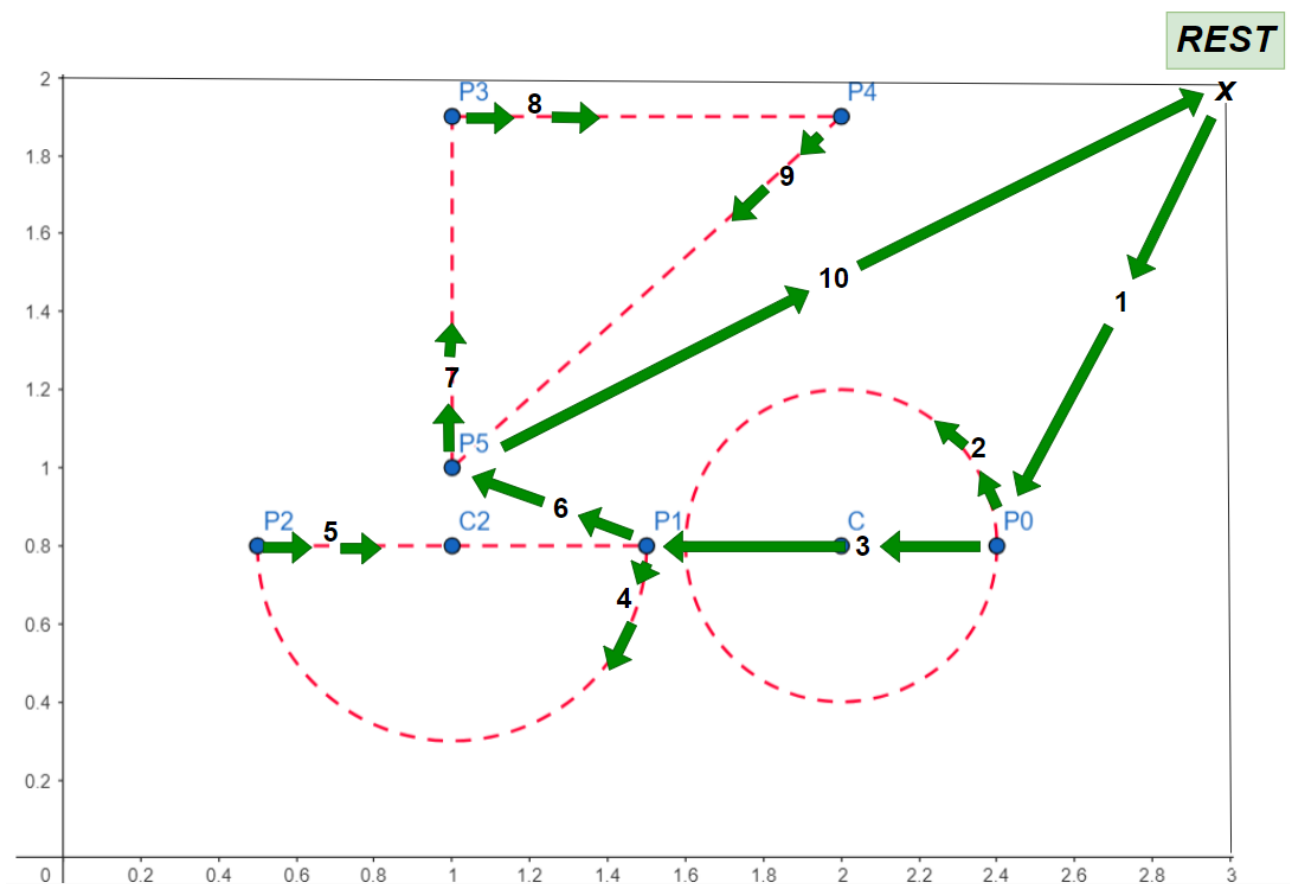


Fig. 3.10: Sequence Movements of the End Effector

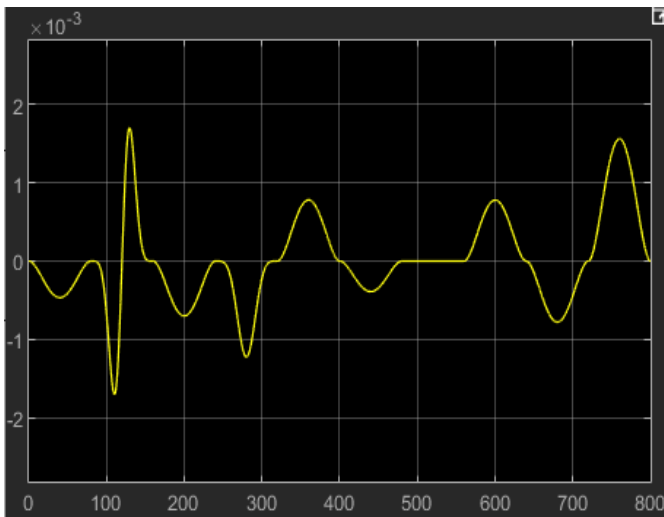
So far, we can assert the Control Unit has satisfied the requirements in nominal conditions. Now we have to check its behaviour when there is an uncertainty of the parameters of 10 %. Thus, there are four cases to analyse:

1. $J=1.1*J$ and $B=1.1*B$
2. $J=0.9*J$ and $B=0.9*B$
3. $J=1.1*J$ and $B=0.9*B$
4. $J=0.9*J$ and $B=1.1*B$

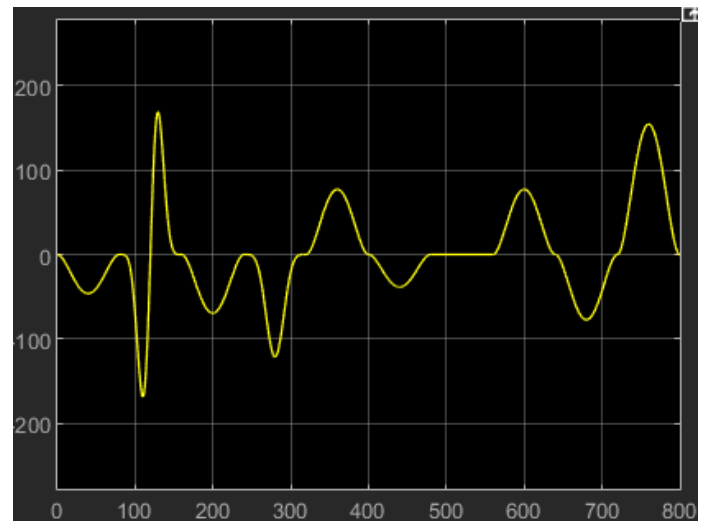
The following results are referring to the x motor, for the y case they are attached in the appendix:

$J=1.1*J$ and $B=1.1*B$

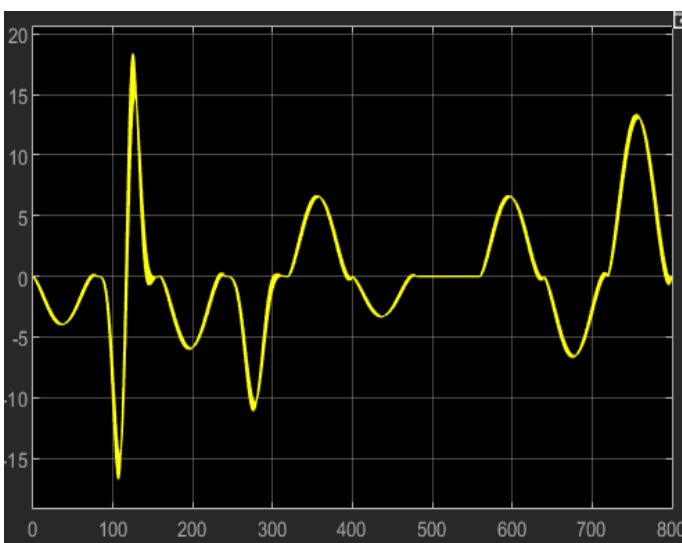
X Linear Position Error



X Voltage



X Current



X Power

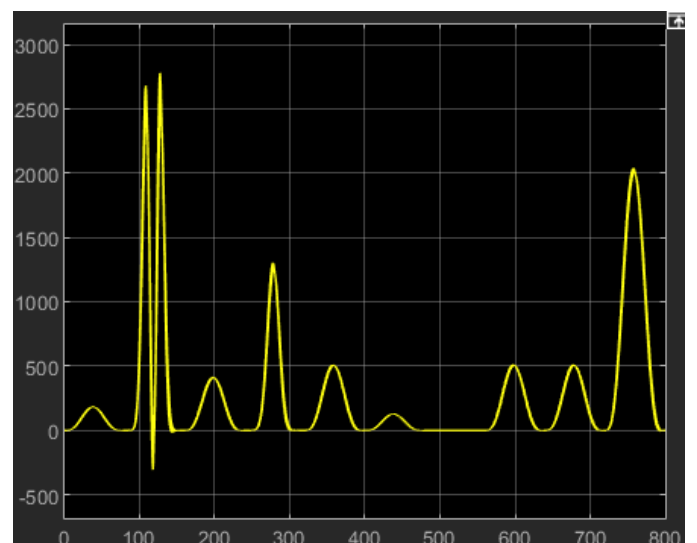
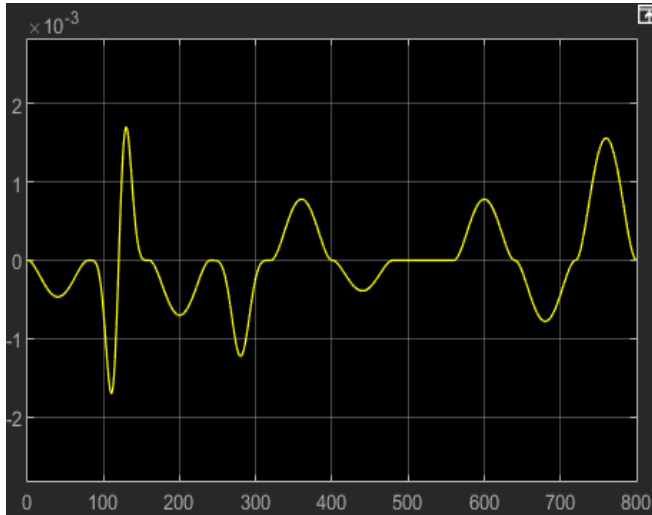


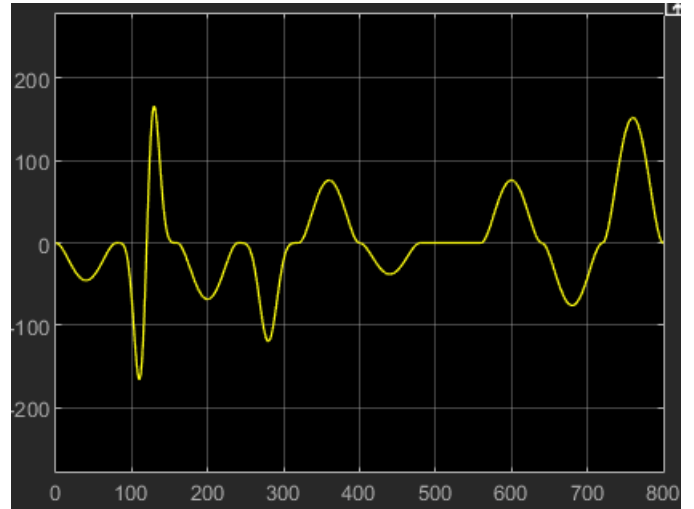
Fig. 3.11: X DC Motor Results with $J=1.1*J$ and $B=1.1*B$ (Simulink)

$$J=0.9*J \text{ and } B=0.9*B$$

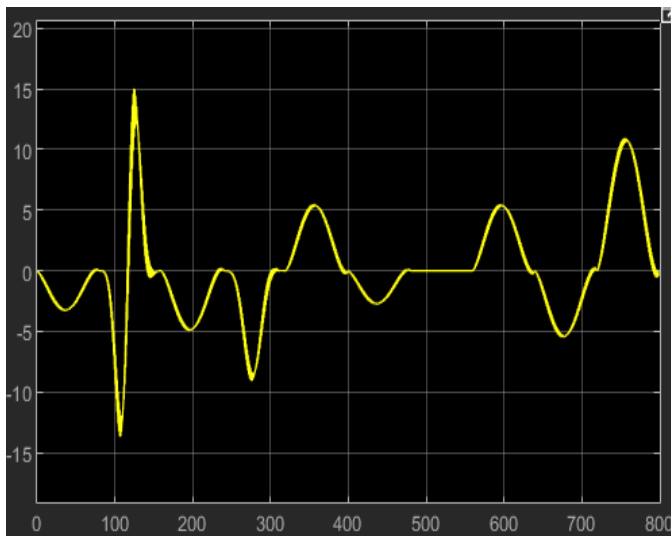
X Linear Position Error



X Voltage



X Current



X Power

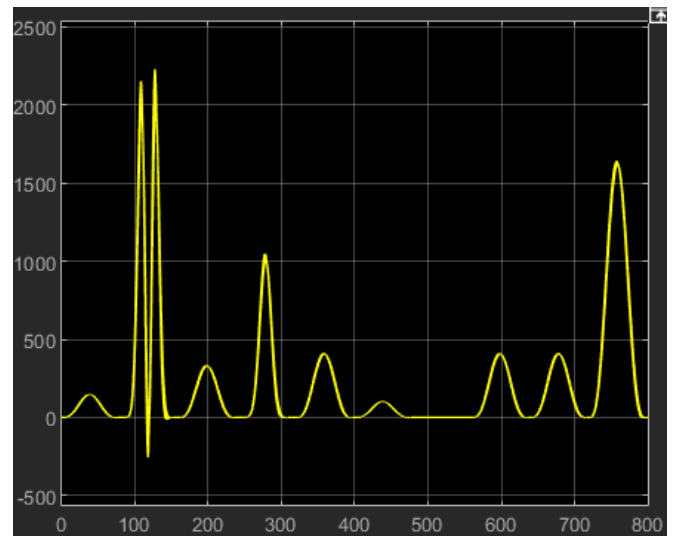
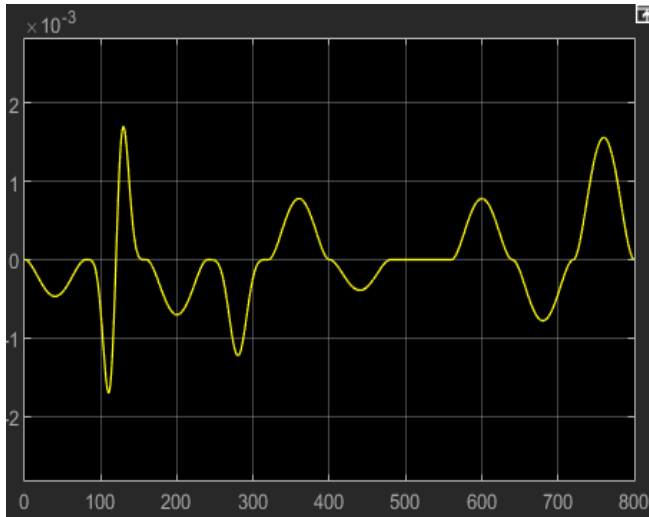


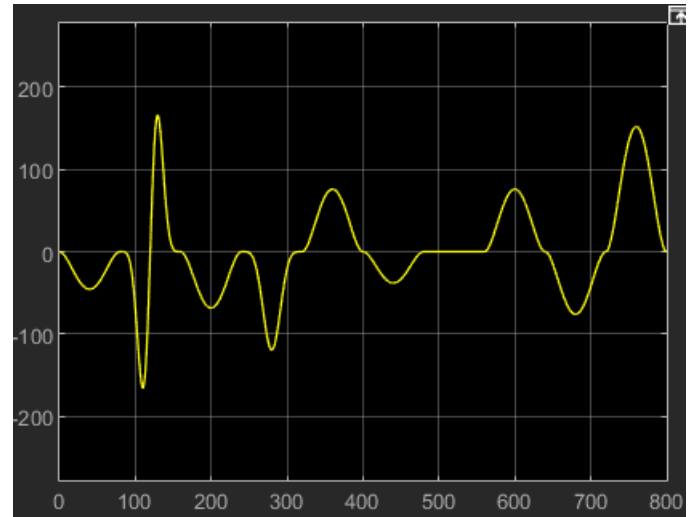
Fig. 3.12: X DC Motor Results with $J=0.9*J$ and $B=0.9*B$ (Simulink)

$$J=1.1*J \text{ and } B=0.9*B$$

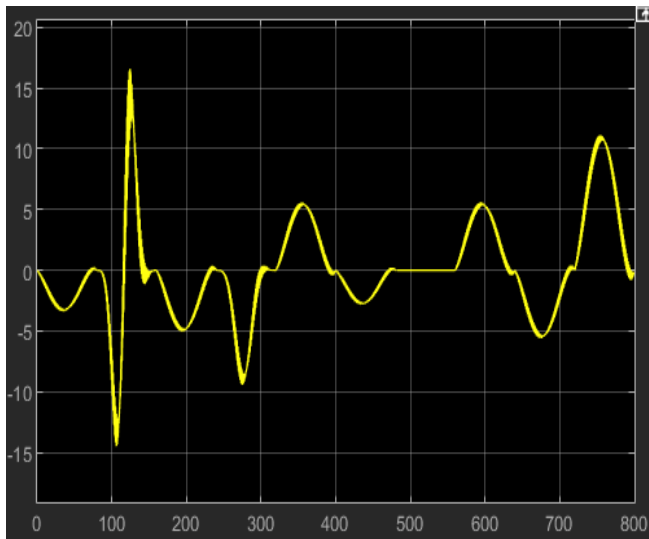
X Linear Position Error



X Voltage



X Current



X Power

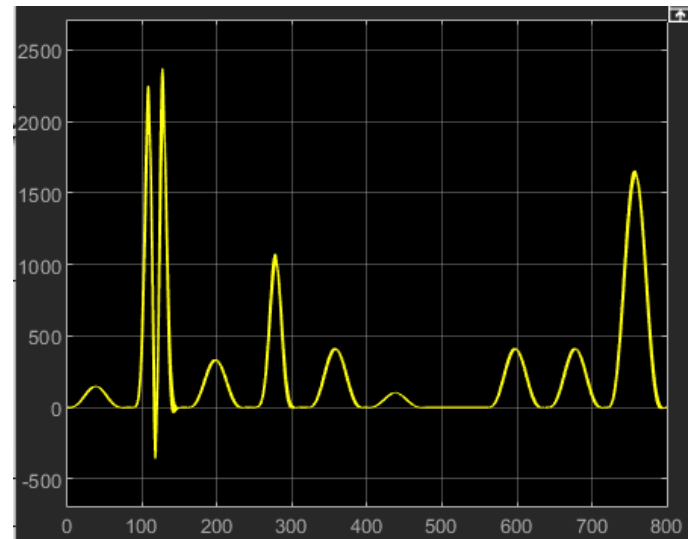
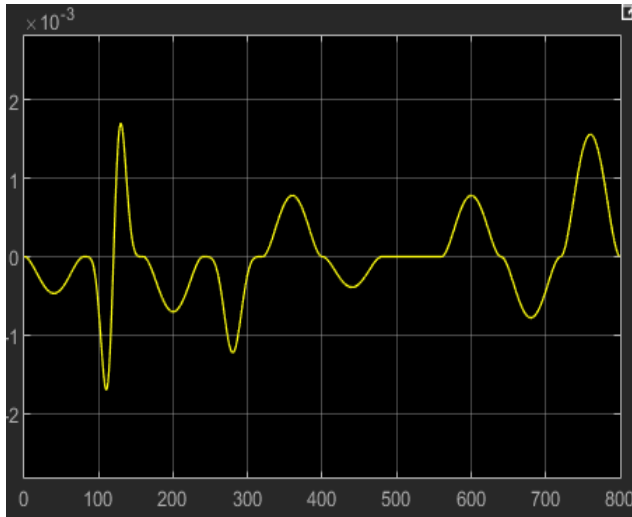


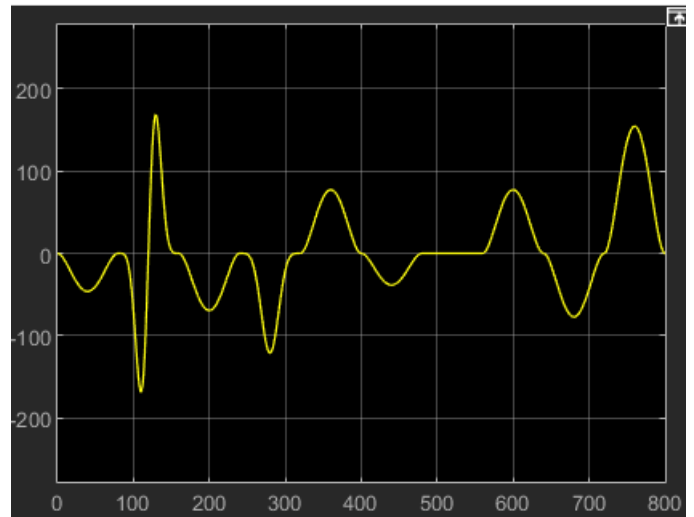
Fig. 3.13: X DC Motor Results with $J=1.1*J$ and $B=0.9*B$ (Simulink)

$$J=0.9*J \text{ and } B=1.1*B$$

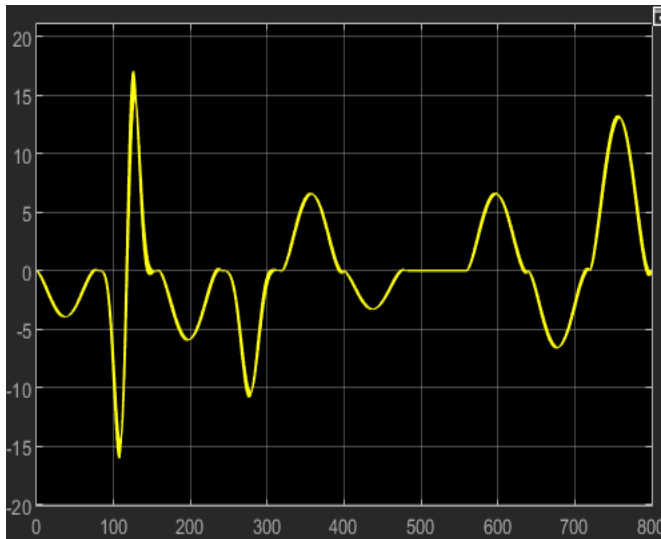
X Linear Position Error



X Voltage



X Current



X Power

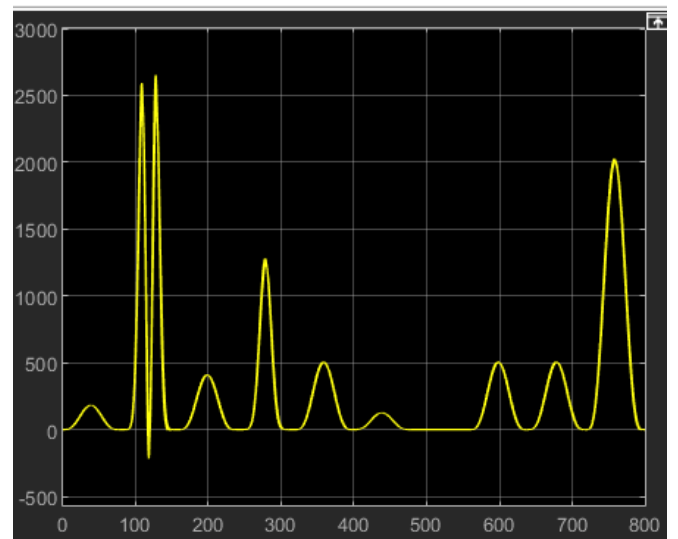


Fig. 3.14: X DC Motor Results with $J=0.9*J$ and $B=1.1*B$ (Simulink)

We can conclude that the motor has satisfied the requirements also with uncertainty of parameters of 10 %

FINITE STATE MACHINE

Last requirement is the development of a finite state machine and its possible implementation in Ladder language. We can note that the machine can be represented by 2 states: it is in a cut condition on the paths characterizing the shapes while it is in a non-cut condition during the other movements. The change of the state is logic to be referred to a control signal **C** of this type:

- C=1 when the laser of the machine is active
- C=0 when the laser of the machine is deactivated

Thus, the finite state machine for the model can be the following:

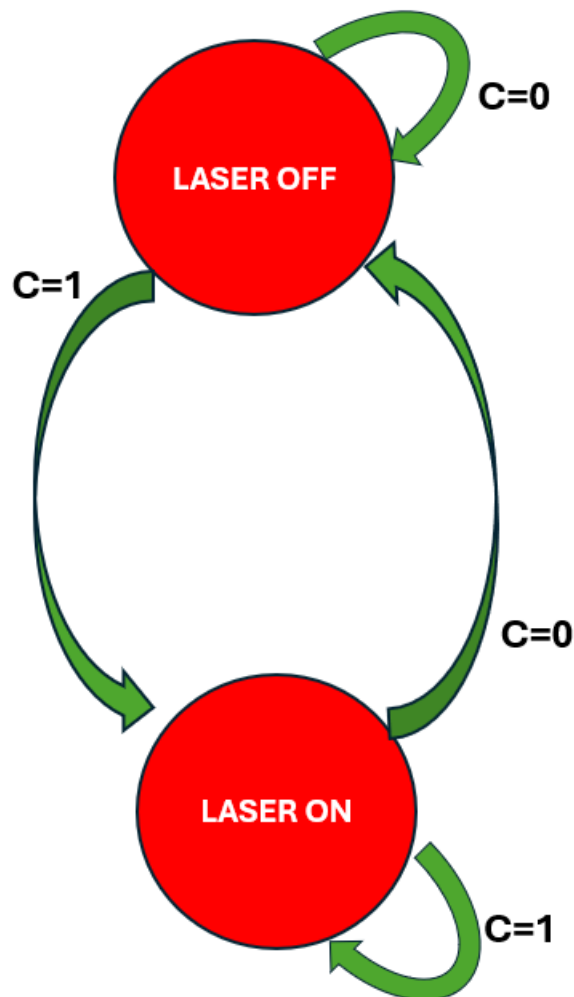


Fig. 4.1: Finite State Machine Diagram

The implementation in Ladder language is simple because of we need only 1 switch associated with the output:

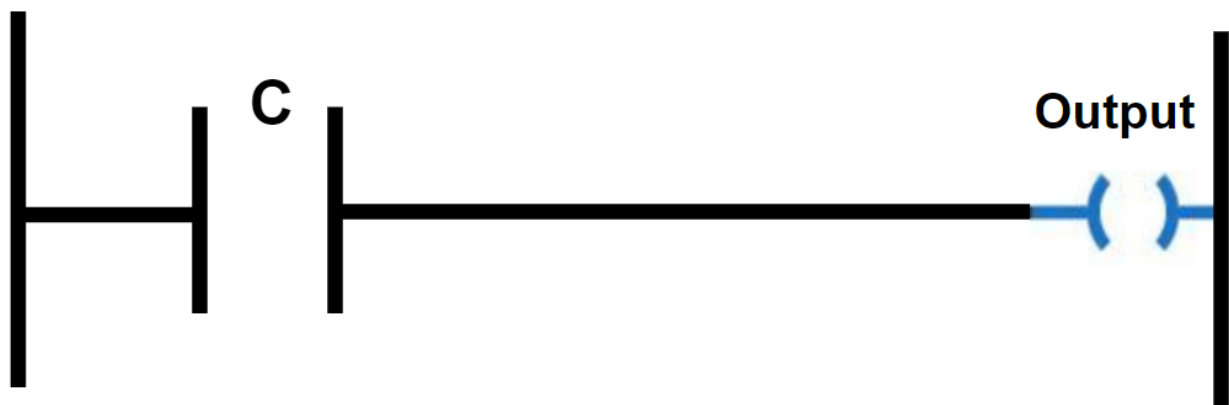


Fig. 4.2: Ladder Scheme

APPENDIX

Y Reference Generator Code:

```
%% Y Reference Generator:
Cy=0.8;%center of the circle [y coordinate]
P0=0.8;%%point of the circle [y coordinate]
r=0.4;%radius of the circle [m]
P1=0.8;%extreme point of the half circle [y coordinate]
P2=0.8;%extreme point of the half circle [y coordinate]
C2y=0.8;%center of the half circle [y coordinate]
r2=0.5;%radius of the half circle [m]
P5=1;%vertex of the triangle [y coordinate]
P3=1.9;%vertex of the triangle [y coordinate]
P4=1.9;%vertex of the triangle [y coordinate]
yrest=2;%rest position [y coordinate]
deltat=80;%time interval for each path

    %Segment from rest to P0: Laser Off
    t0=0;t1=deltat;
    sigma=[0:0.1/80:1];
    t_1=t0+sigma*(t1-t0);
    y0=yrest;y1=P0;
    sigma_star=(t_1-t0)/(t1-t0);
    lambda_star=poly5(sigma_star);
    y_1=y0+lambda_star*(y1-y0);
    t=t_1;
    %Circular shape: Laser On
    sigma=[0.1/80:0.1/80:1];
    t2=0;t3=deltat;
    t_2=t2+sigma*(t3-t2);
    sigma_star=(t_2-t2)/(t3-t2);
    lambda_star=poly5(sigma_star);
    y_2=Cy+r*sin(lambda_star*2*pi);
```

```

t=[t,t_2+t(end)];
%Segment from P0 to P1: Laser Off
t4=0;t5=deltat;
t_3=t4+sigma*(t5-t4);
sigma_star=(t_3-t4)/(t5-t4);
y2=P0;y3=P1;
lambda_star=poly5(sigma_star);
y_3=y2+lambda_star*(y3-y2);
t=[t,t_3+t(end)];
%Half circular shape:Laser On
t6=0;t7=deltat;
t_4=t6+sigma*(t7-t6);
sigma_star=(t_4-t6)/(t7-t6);
lambda_star=poly5(sigma_star);
y_4=C2y-r2*sin(lambda_star*pi);
t=[t,t_4+t(end)];
%Segment from P2 to P1:Laser On
t8=0;t9=deltat;
t_5=t8+sigma*(t9-t8);
sigma_star=(t_5-t8)/(t9-t8);
y4=P2;y5=P1;
lambda_star=poly5(sigma_star);
y_5=y4+lambda_star*(y5-y4);
t=[t,t_5+t(end)];
%Segment from P1 to P5: Laser OFF
t10=0;t11=deltat;
t_6=t10+sigma*(t11-t10);
sigma_star=(t_6-t10)/(t11-t10);
y6=P1;y7=P5;
lambda_star=poly5(sigma_star);

```

```

y_6=y6+lambda_star*(y7-y6);
t=[t,t_6+t(end)];
%Segment from P5 to P3: Laser ON
t12=0;t13=deltat;
t_7=t12+sigma*(t13-t12);
sigma_star=(t_7-t12)/(t13-t12);
y8=P5;y9=P3;
lambda_star=poly5(sigma_star);
y_7=y8+lambda_star*(y9-y8);
t=[t,t_7+t(end)];
%Segment from P3 to P4: Laser ON
t14=0;t15=deltat;
t_8=t14+sigma*(t15-t14);
sigma_star=(t_8-t14)/(t15-t14);
y10=P3;y11=P4;

lambda_star=poly5(sigma_star);
y_8=y10+lambda_star*(y11-y10);
t=[t,t_8+t(end)];
%Segment from P4 to P5: Laser ON
t16=0;t17=deltat;
t_9=t16+sigma*(t17-t16);
sigma_star=(t_9-t16)/(t17-t16);
y12=P4;y13=P5;
lambda_star=poly5(sigma_star);
y_9=y12+lambda_star*(y13-y12);
t=[t,t_9+t(end)];
%Segment from P5 to rest: Laser OFF
t18=0;t19=deltat;
t_10=t18+sigma*(t19-t18);
sigma_star=(t_10-t18)/(t19-t18);

y14=P5;y15=yrest;
lambda_star=poly5(sigma_star);
y_10=y14+lambda_star*(y15-y14);
t=[t,t_10+t(end)];
y=[y_1,y_2,y_3,y_4,y_5,y_6,y_7,y_8,y_9,y_10];

```

Results of Y Motor in Nominal Conditions:

Y Linear Position Error

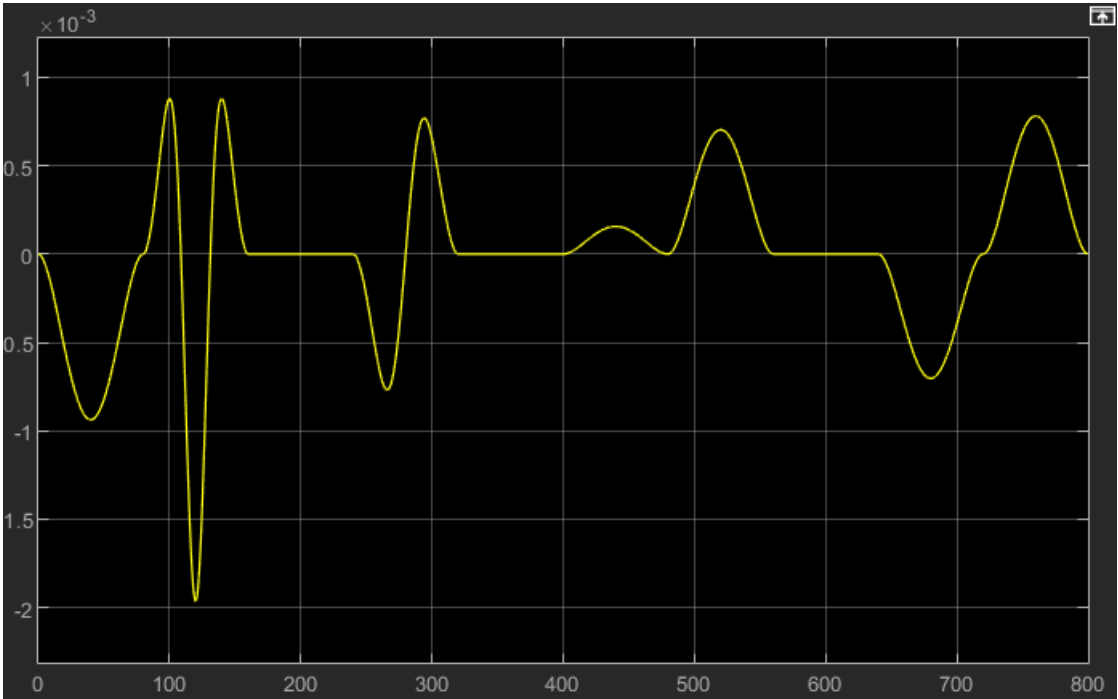


Fig. A.1: Y Linear Position Error (Simulink)

Y Voltage

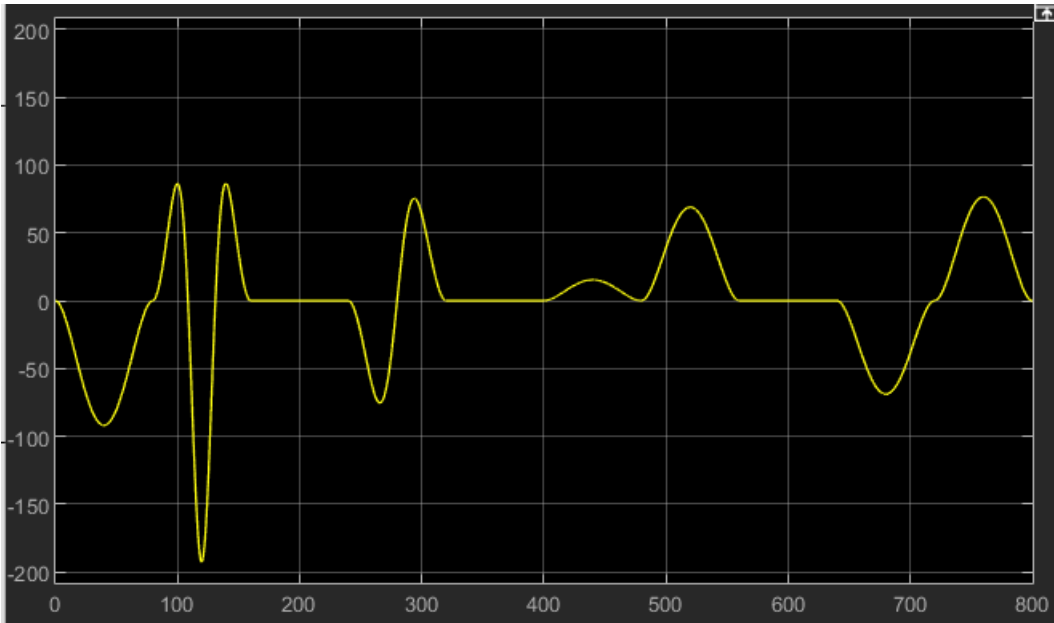


Fig. A.2: Y DC Motor Voltage (Simulink)

Y Current

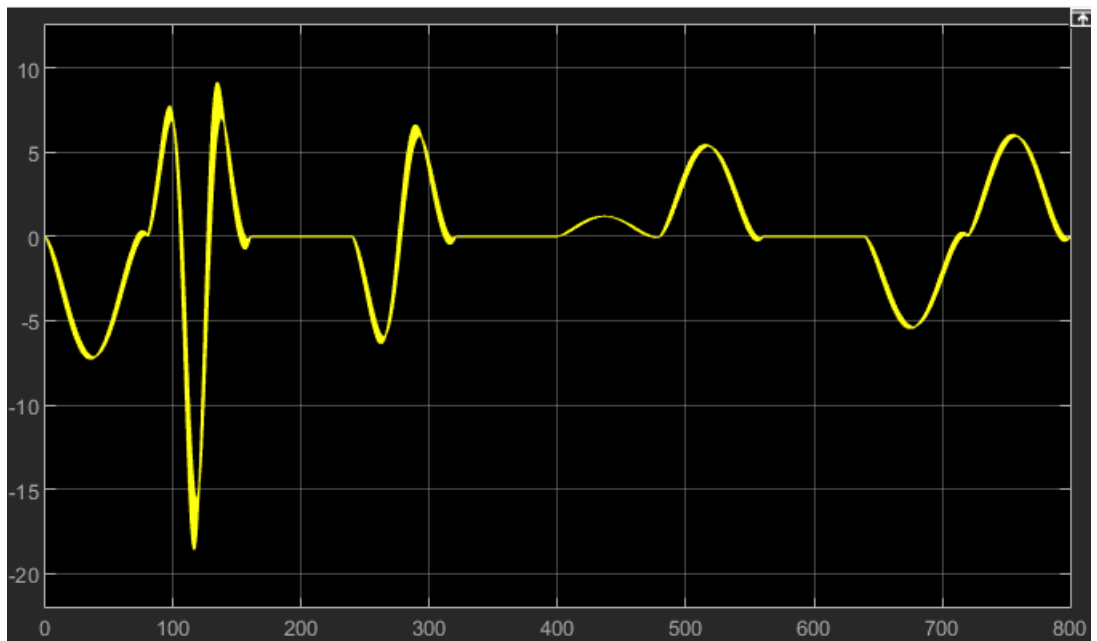


Fig. A.3: Y DC Motor Current (Simulink)

Y Power

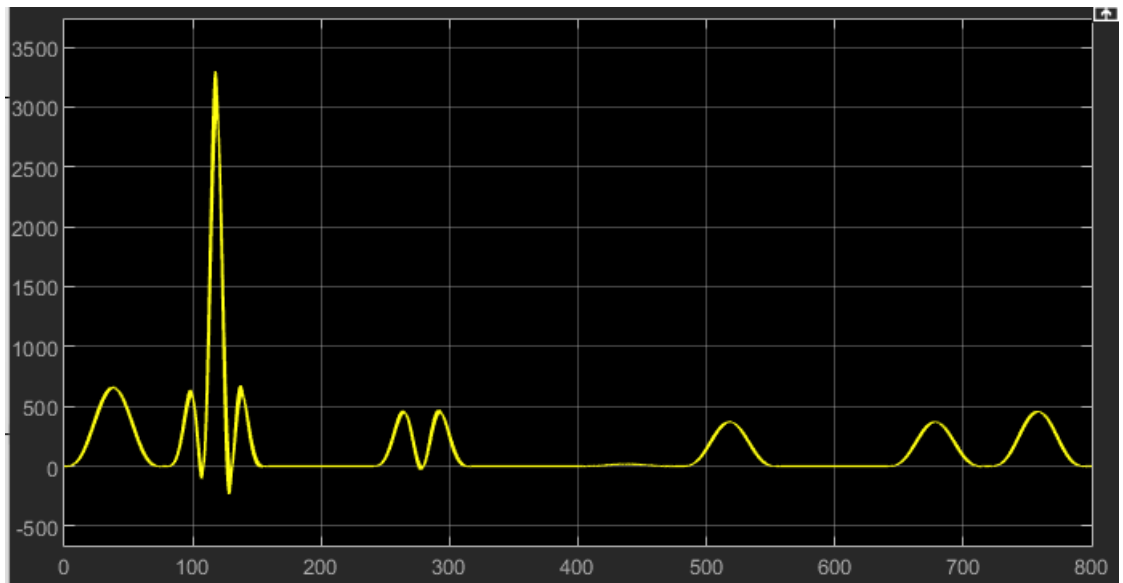
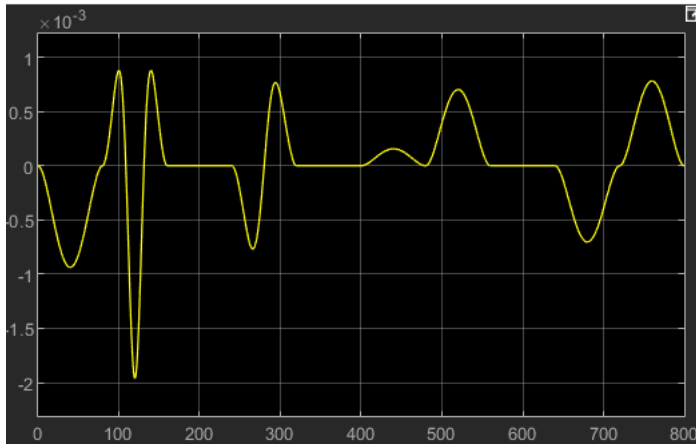


Fig. A.4: Y DC Motor Power (Simulink)

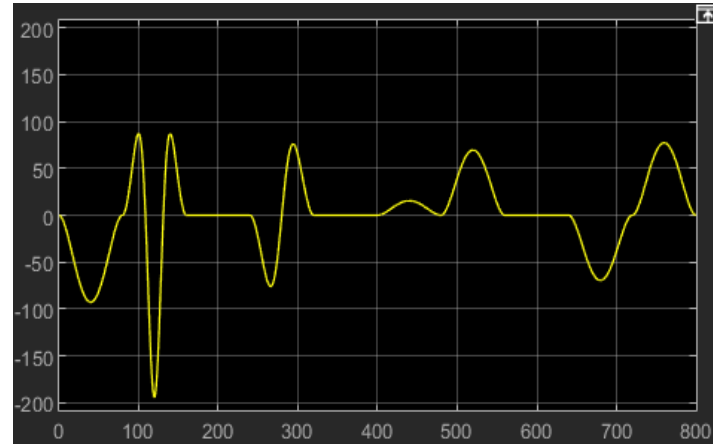
Results of Y Motor with Uncertainty of Parameters of 10%:

$$J=1.1*J \text{ and } B=1.1*B$$

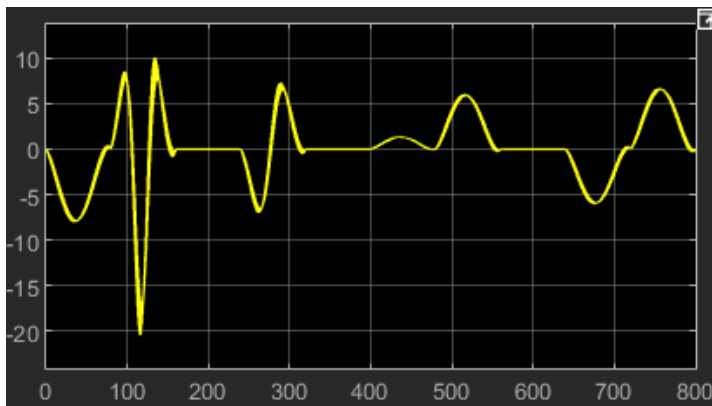
Y Linear Error



Y Voltage



Y Current



Y Power

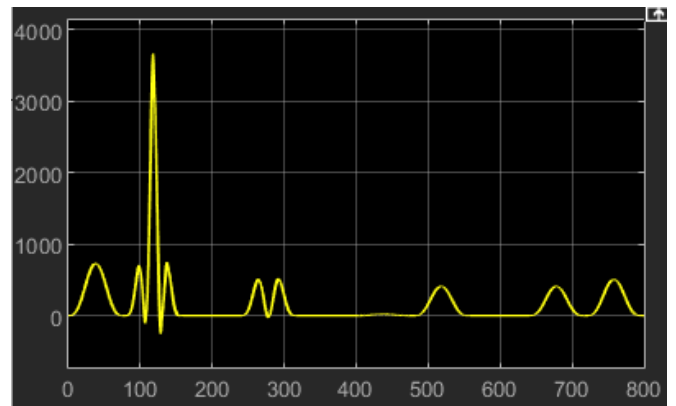
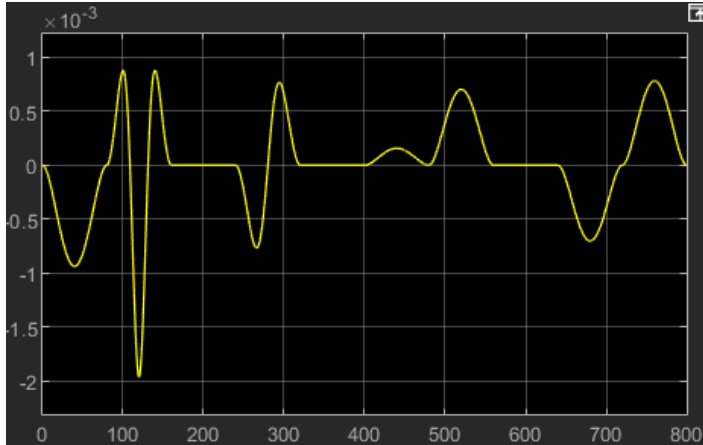


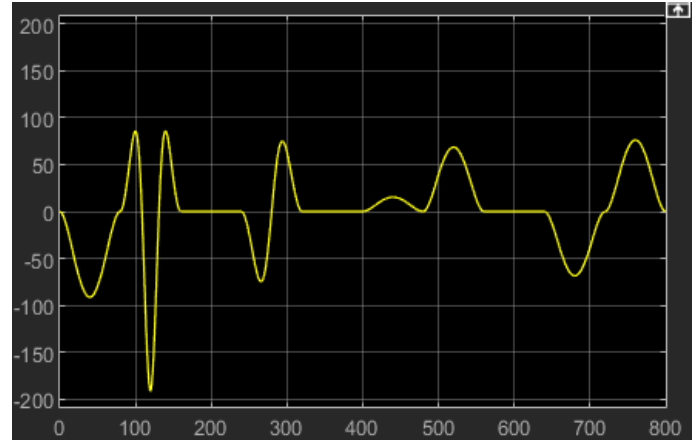
Fig. A.5: Y DC Motor Results with $J=1.1*J$ and $B=1.1*B$ (Simulink)

$$J=0.9*J \text{ and } B=0.9*B$$

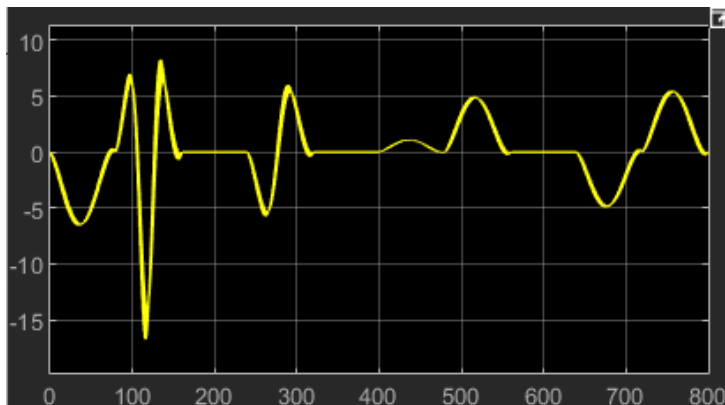
Y Linear Error



Y Voltage



Y Current



Y Power

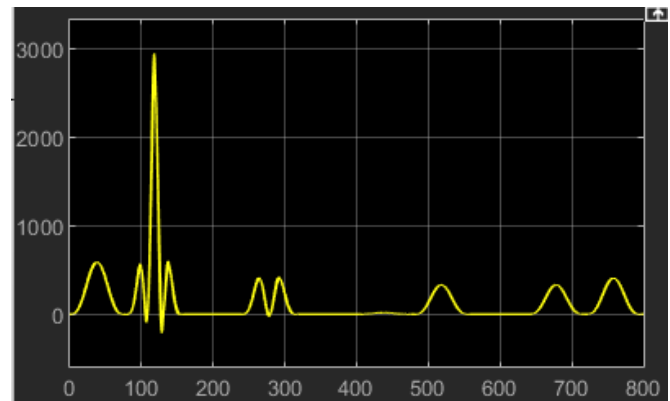
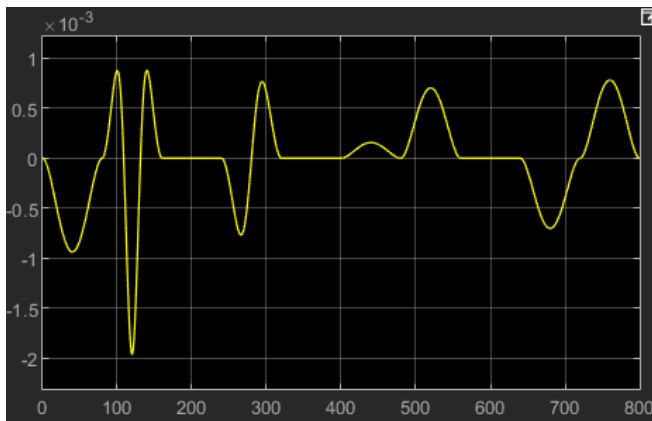


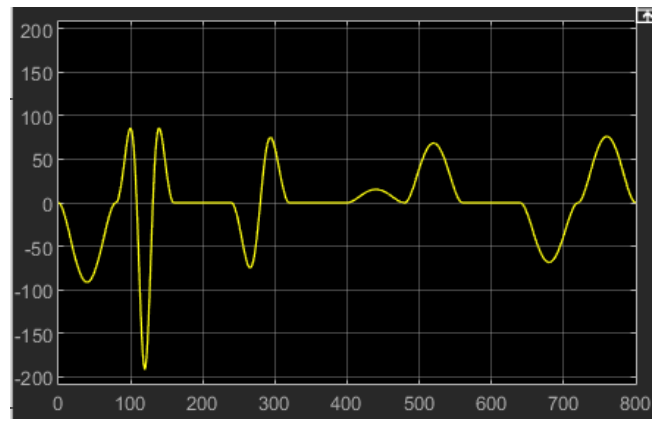
Fig. A.6: Y DC Motor Results with $J=0.9*J$ and $B=0.9*B$ (Simulink)

$J=1.1*J$ and $B=0.9*B$

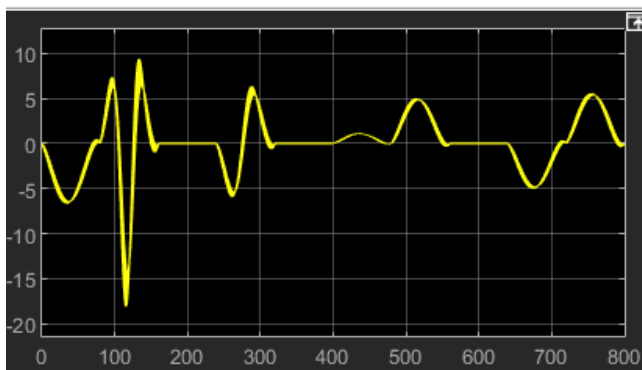
Y Linear Error



Y Voltage



Y Current



Y Power

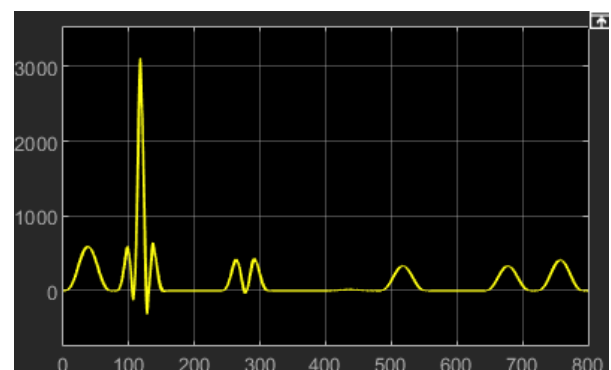
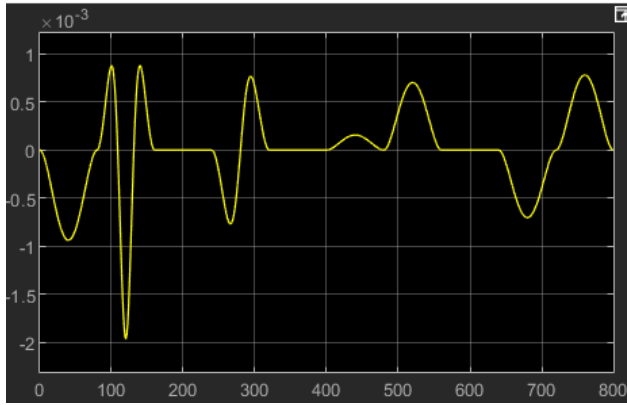


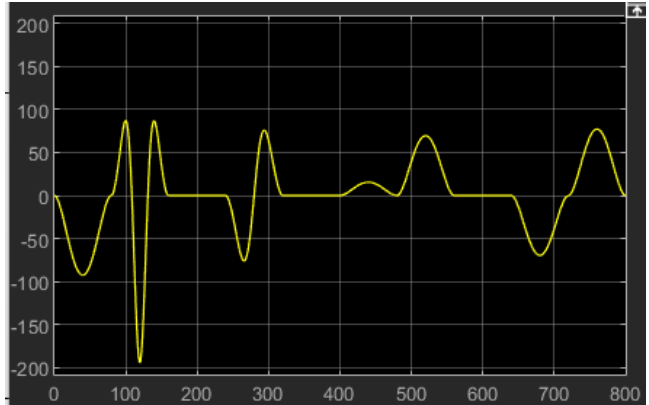
Fig. A.7: Y DC Motor Results with $J=1.1*J$ and $B=0.9*B$ (Simulink)

$$J=0.9*J \text{ and } B=1.1*B$$

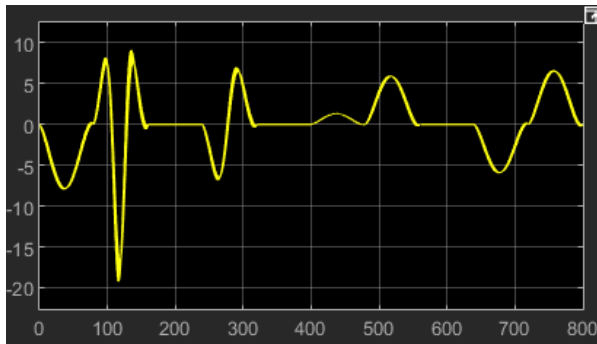
Y Linear Error



Y Voltage



Y Current



Y Power

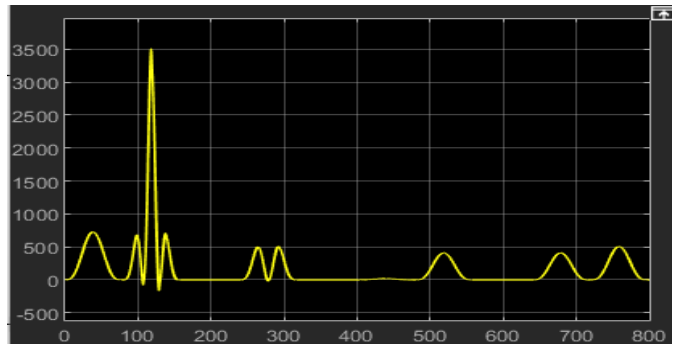


Fig. A.8: Y DC Motor Results with $J=0.9*J$ and $B=1.1*B$ (Simulink)